

Học sâu và ứng dụng

Mở đầu

Giảng viên: TS. Phạm Đình Phong

Email: phongpd@utc.edu.vn

Phone: 0972481813

Giới thiệu học phần

- **Mục tiêu của học phần**

- **Kiến thức:**

- Cung cấp cho sinh viên các kiến thức, kỹ năng về mạng nơ-ron (neural network), phương pháp huấn luyện deep learning (học sâu), cũng như quy trình xây dựng một hệ thống deep learning.
 - Giới thiệu một số kiến trúc mạng neuron phổ biến như: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), và một số biến thể của các kiến trúc này.

- **Kỹ năng:**

- Sinh viên sẽ có các kỹ năng xây dựng, tinh chỉnh, thực nghiệm, đánh giá hệ thống deep learning.
 - Sinh viên có kỹ năng lựa chọn mô hình học sâu, biết cách xây dựng và huấn luyện mô hình để nhận dạng đối tượng.
 - Có khả năng làm việc nhóm để khảo sát, hình thành ý tưởng và thiết kế hệ thống nhận dạng đối tượng từ ảnh/video.

Giới thiệu học phần

Thành phần đánh giá	Hình thức đánh giá	Tiêu chí đánh giá	Tỷ lệ (%)
Đánh giá quá trình	1. Điểm danh	Chuyên cần	5%
	2. Bài tập, thực hành	Sử dụng thành thạo các kiến thức về mô hình học sâu, các kỹ thuật xử lý ảnh để gán nhãn, phân tích và thiết kế mô hình	5%
	3. Bài kiểm tra giữa kỳ	Sử dụng thành thạo một số thư viện học máy, xử lý ảnh để viết chương trình giải quyết một vấn đề đơn giản	30%
Đánh giá kết thúc học phần	Bài thi cuối kỳ dưới dạng thi trắc nghiệm, thực hành, tự luận	Sử dụng thành thạo các kiến thức về cấu trúc và thành phần của một mô hình học sâu và cơ chế hoạt động của chúng, thiết kế một mô hình để xây dựng hệ thống nhận dạng	60%

Giới thiệu học phần

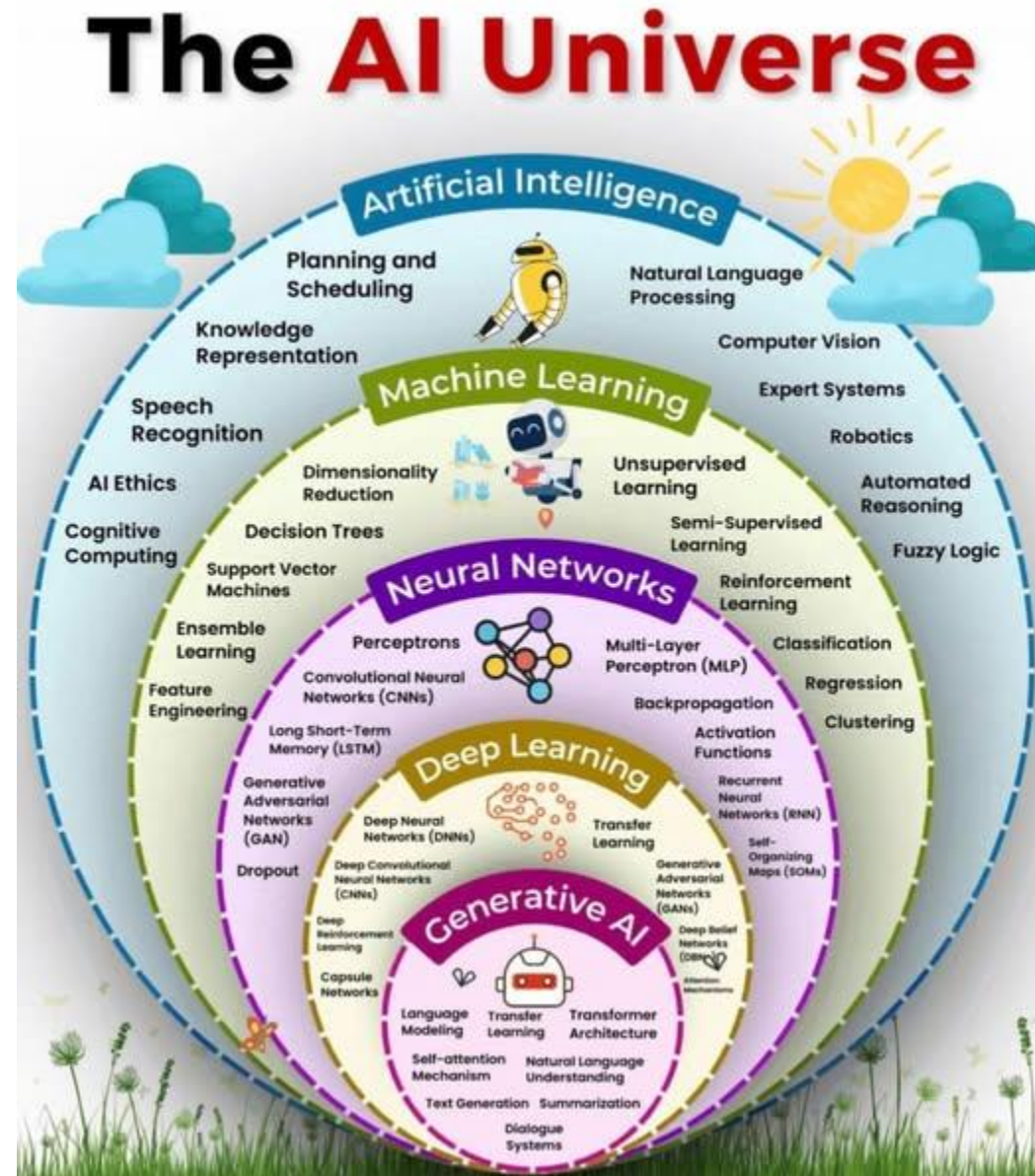
- **Tài liệu tham khảo**

- Deep learning cơ bản, Nguyễn Thanh Tuấn, 2020
- Deep learning, Ian Goodfellow, Yoshua Bengio, Aaron Courville, The MIT Press, 2016.
- Dive into Deep Learning, Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola, 2021
- Fundamentals of Deep Learning, Nikhil Buduma, O'Reilly Media, Inc., 2017

Giới thiệu học sâu (Deep learning)

- Trí tuệ nhân tạo (AI) hiện diện trong mọi lĩnh vực của đời sống con người, từ kinh tế, giáo dục, y khoa cho đến những công việc nhà, giải trí hay thậm chí là trong quân sự.
- Những ứng dụng nổi bật trong việc phát triển AI đến từ nhiều lĩnh vực để giải quyết nhiều vấn đề khác nhau.
- Những đột phá phần lớn đến từ học sâu (Deep Learning), một nhánh nhỏ của AI, đang dần giải quyết các công việc từ đơn giản đến phức tạp.
- Học sâu đã giúp máy tính thực thi những việc tưởng chừng như không thể vào 15 năm trước như: phân loại cả ngàn vật thể khác nhau trong các bức ảnh, tự tạo chú thích cho ảnh, bắt chước giọng nói và chữ viết của con người, giao tiếp với con người, hay thậm chí cả sáng tác thơ văn, phim, ảnh, âm nhạc.

Giới thiệu học sâu (Deep learning)



Giới thiệu học sâu (Deep learning)

- Lý do học sâu được nhắc đến rất nhiều như một xu hướng mới của cuộc cách mạng AI
 - **Bùng nổ dữ liệu:** Học sâu khai thác được dữ liệu lớn (Big Data) với độ chính xác cao hơn hẳn so với các phương pháp học máy (Machine Learning) khác trên tập dữ liệu đặc biệt là đối với ảnh.
 - **Phần cứng phát triển:** Sự xuất hiện của GPU GTX 10 series của NVIDIA năm 2014 với hiệu năng tính toán cao cũng như giá thành rẻ có thể tiếp cận với hầu hết với mọi người dẫn đến việc nghiên cứu học sâu không còn là những bài toán chỉ được nghiên cứu trong các phòng lab đắt tiền của các trường Đại học danh giá và các công ty lớn

Giới thiệu học sâu (Deep learning)

- Theo thống kê trên trang [paperswithcode](#) hiện có 16 tasks lớn mà học máy có thể thực hiện trong đó có tới trên 8 tasks học sâu đạt kết quả SOTA phải kể đến như:
 - Computer Vision
 - Natural Language Processing
 - Medical
 - Methodology
 - Speech
 - Time Series
 - Audio
 - Music

Thư viện Numpy

- Numpy là thư viện viết bằng C có giao diện với Python
- Numpy cộng hưởng hai ưu điểm của hai ngôn ngữ: nhanh như C và đơn giản như Python.
- Giúp cài đặt các thuật toán học máy bằng Python

Thư viện Numpy

- Mảng (Array)
 - Gồm các phần tử có cùng kiểu giá trị, chỉ số không âm được bắt đầu từ 0
 - Số chiều được gọi là rank của mảng Numpy
 - shape của mảng là một tuple các số nguyên đưa ra kích thước của mảng theo mỗi chiều.

Thư viện Numpy

- Ví dụ:

```
import numpy as np
a = np.array([1, 2, 3]) # Tạo array 1 chiều
print(type(a))          # Prints "<class 'numpy.ndarray'>"
print(a.shape)          # Prints "(3,)"
print(a[0], a[1], a[2]) # Prints "1 2 3"
a[0] = 5                # Thay đổi phần tử ở vị trí số 0
print(a)                # Prints "[5, 2, 3]"
b = np.array([[1,2,3],[4,5,6]]) # Tạo array 2 chiều
print(b.shape)          # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0]) # Prints "1 2 4"
```

Thư viện Numpy

- Có thể khởi tạo giá trị mặc định bằng một số hàm:

```
import numpy as np
a = np.zeros((2,2))      # Tạo array với tất cả các phần tử 0
print(a)                 # Prints "[[ 0.  0.]
                          # [ 0.  0.]]"
b = np.ones((1,2))       # Tạo array với các phần tử 1
print(b)                 # Prints "[[ 1.  1.]]"
c = np.full((2,2), 7)    # Tạo array với các phần tử 7
print(c)                 # Prints "[[ 7.  7.]
                          #          [ 7.  7.]]"
d = np.eye(2)            # Tạo identity matrix kích thước 2*2
print(d)                 # Prints "[[ 1.  0.]
                          #          [ 0.  1.]]"
```

Thư viện Numpy

- **Array indexing:** Tương tự như list, numpy array cũng có thể slice. Vì numpy array có nhiều chiều, nên khi dùng slice phải chỉ định rõ chiều nào.

```
import numpy as np
# Tạo array 2 chiều với kích thước (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
# Dùng slice để lấy ra subarray gồm 2 hàng đầu tiên (1 & 2) và 2 cột (2 & 3).
# Output là array kích thước 2*2
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]
print(a[0, 1])    # Prints "2"
a[0, 1] = 77      # Sửa phần tử trong array
print(a[0, 1])    # Prints "77"
```

Thư viện Numpy

- Có thể dùng các chỉ số với slice index. Tuy nhiên số chiều array sẽ giảm đi.

```
import numpy as np
# Tạo array 2 chiều kích thước (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
row_r1 = a[1, :]      # Lấy ra hàng thứ 2 trong a, output array 1 chiều
row_r2 = a[1:2, :]    # Lấy ra hàng thứ 1&2 trong a, output array 2 chiều
print(row_r1, row_r1.shape) # Prints "[5 6 7 8] (4,)"
print(row_r2, row_r2.shape) # Prints "[[5 6 7 8]] (1, 4)"
```

Thư viện Numpy

- Các phép tính trên array

```
import numpy as np
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)
# Tính tổng
# [[ 6.0  8.0]
# [10.0 12.0]]
print(x + y)
print(np.add(x, y))
# Phép trừ
# [[-4.0 -4.0]
# [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))
```

Thư viện Numpy

- Các phép tính trên array

```
import numpy as np
# Phép nhân element-wise
# [[ 5.0 12.0]
# [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))
# Phép chia element-wise
# [[ 0.2 0.33333333]
# [ 0.42857143 0.5 ]]
print(x / y)
print(np.divide(x, y))
# Tính căn bậc hai
# [[ 1. 1.41421356]
# [ 1.73205081 2. ]]
print(np.sqrt(x))
```


Thư viện Numpy

- Phép toán `*` dùng để nhân element-wise chứ không phải nhân ma trận thông thường. Thay vào đó dùng `np.dot` để nhân 2 ma trận.

```
import numpy as np
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])
v = np.array([9,10])
w = np.array([11, 12])
# Nhân ma trận, output số 219
print(v.dot(w))
print(np.dot(v, w))
# Nhân ma trận; output ma trận 2*2
# [[19 22]
#  [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Thư viện Numpy

- **Broadcasting**: là một kĩ thuật cho phép numpy làm việc với các array có shape khác nhau khi thực hiện các phép toán.

```
import numpy as np
# Cộng vector v với mỗi hàng của ma trận x, kết quả lưu ở ma trận y.
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = np.empty_like(x) # Tạo 1 array có chiều giống x
# Dùng loop để vector v với mỗi hàng của ma trận
for i in range(4):
    y[i, :] = x[i, :] + v
print(y)
# Kết quả của y
# [[ 2  2  4]
#  [ 5  5  7]
#  [ 8  8 10]
#  [11 11 13]]
```

Thư viện Numpy

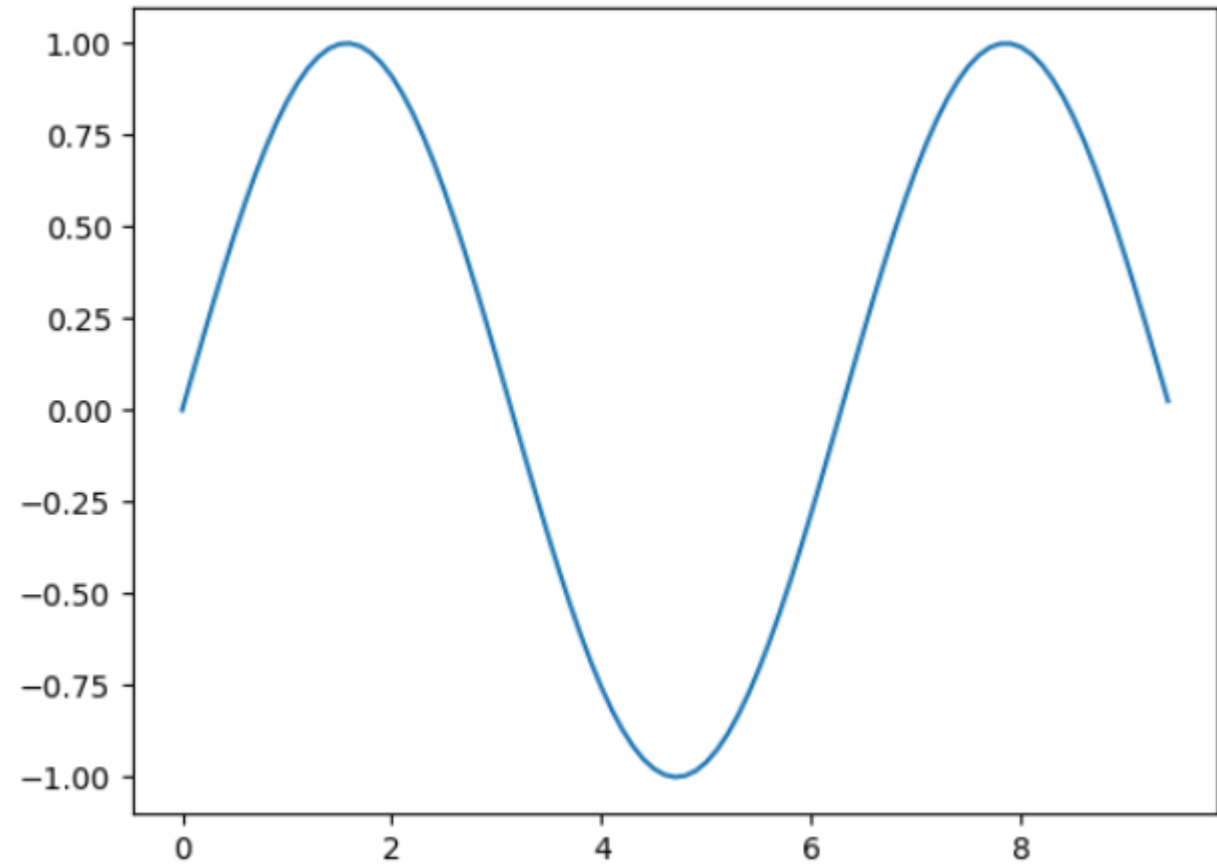
- **Broadcasting**: là một kĩ thuật cho phép numpy làm việc với các array có shape khác nhau khi thực hiện các phép toán.

```
import numpy as np
# Cộng vector v với mỗi hàng của ma trận x, kết quả lưu ở ma trận y.
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = np.empty_like(x) # Tạo 1 array có chiều giống x
# Cộng kiểu broadcasting trong python
print(x + v)
# Kết quả
# [[ 2  2  4]
#  [ 5  5  7]
#  [ 8  8 10]
#  [11 11 13]]
# 2 kết quả cho ra giống nhau
```

Thư viện Numpy

- **Matplotlib**: dùng để vẽ đồ thị.
 - Hàm plot để vẽ dữ liệu 2d trong python

```
import numpy as np
import matplotlib.pyplot as plt
# Tính tọa độ x, y cho đồ thị hình sin
x = np.arange(0, 3 * np.pi, 0.1)
y = np.sin(x)
# Plot the points using matplotlib
plt.plot(x, y)
plt.show() # You must call plt.show() to make graphics appear.
```

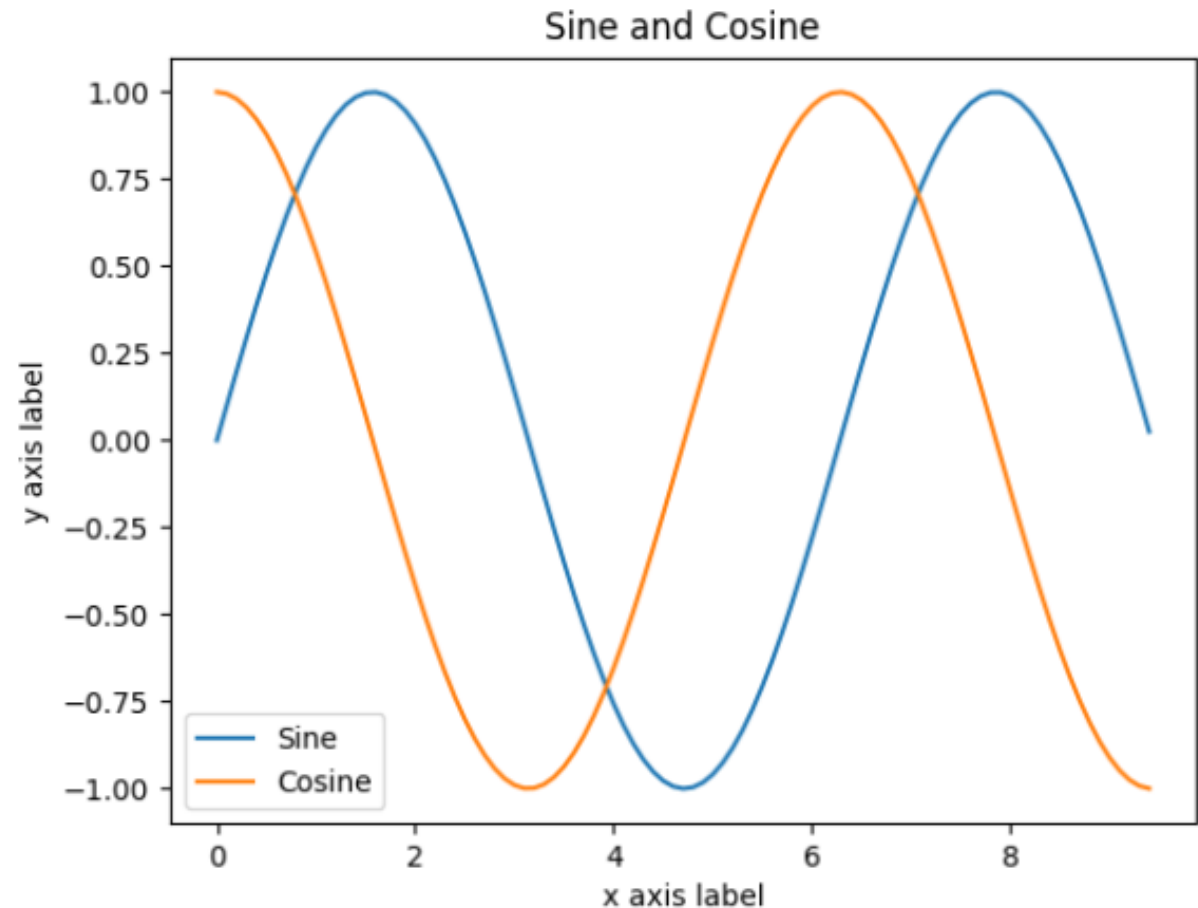


Thư viện Numpy

- **Matplotlib**: dùng để vẽ đồ thị.

- Có thể vẽ nhiều đường trên 1 đồ thị, thêm tên đồ thị, tên trục x, y cũng như tên các đường tương ứng lên đồ thị.

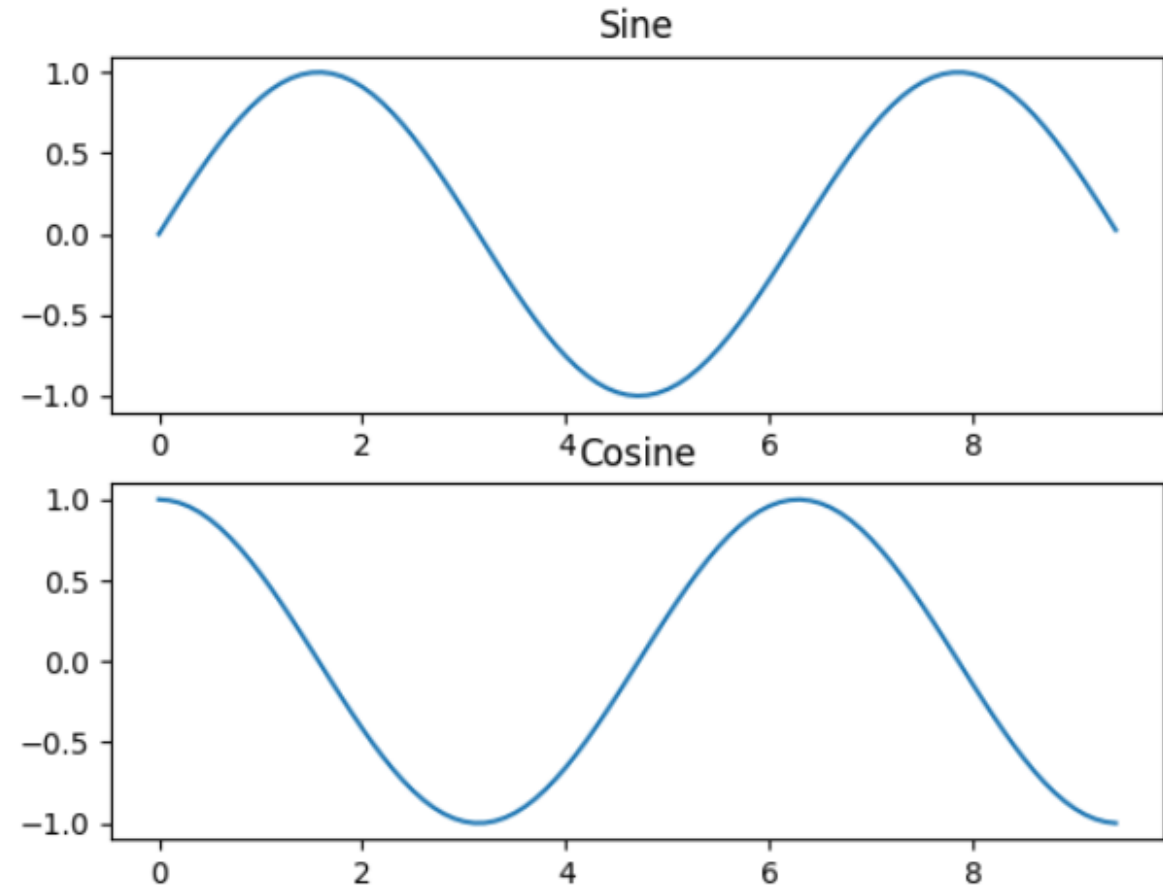
```
import numpy as np
import matplotlib.pyplot as plt
# Tính tọa độ x, y cho đồ thị sin và cos
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)
# Vẽ đồ thị sin, cos dùng matplotlib
plt.plot(x, y_sin)
plt.plot(x, y_cos)
plt.xlabel('x axis label')
plt.ylabel('y axis label')
plt.title('Sine and Cosine')
plt.legend(['Sine', 'Cosine'])
plt.show()
```



Thư viện Numpy

- **Matplotlib**: dùng để vẽ đồ thị.
 - Ta có thể vẽ nhiều subplot và mỗi đồ thị trên 1 subplot riêng.

```
import numpy as np
import matplotlib.pyplot as plt
# Tính tọa độ x, y cho đồ thị sin và cos
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)
# Trên grid kích thước 2*1, vẽ trên grid 1
plt.subplot(2, 1, 1)
plt.plot(x, y_sin)
plt.title('Sine')
# Trên grid kích thước 2*1, vẽ trên grid 2
plt.subplot(2, 1, 2)
plt.plot(x, y_cos)
plt.title('Cosine')
plt.show()
```



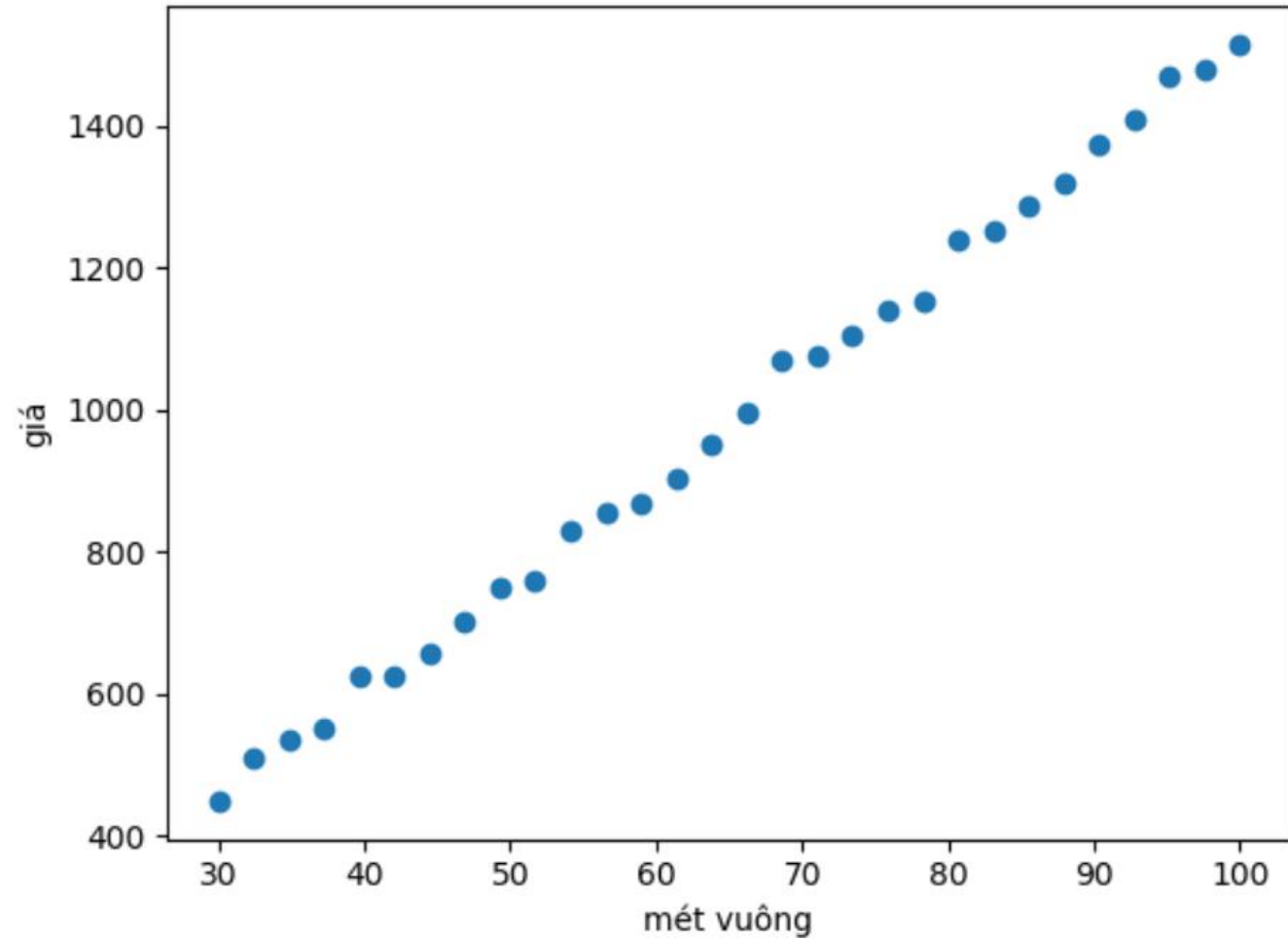
Linear regression (Hồi quy tuyến tính)

- Thuật toán linear regression giải quyết các bài toán có đầu ra là giá trị thực, ví dụ: dự đoán giá nhà, dự đoán giá cổ phiếu, dự đoán tuổi, ...
- **Bài toán:** Bạn làm ở công ty bất động sản, bạn có dữ liệu về diện tích và giá nhà, giờ có một ngôi nhà mới bạn muốn ước tính xem giá ngôi nhà đó khoảng bao nhiêu. Trên thực tế thì giá nhà phụ thuộc rất nhiều yếu tố: diện tích, số phòng, gần trung tâm thương mại, ... nhưng để cho bài toán đơn giản giả sử giá nhà chỉ phụ thuộc vào diện tích căn nhà. Bạn có dữ liệu về diện tích và giá bán của 30 căn nhà như sau:

Linear regression (Hồi quy tuyến tính)

- Bài toán:**

Diện tích(m2)	Giá bán (triệu VNĐ)
30	448.524
32.4138	509.248
34.8276	535.104
37.2414	551.432
39.6552	623.418
...	...

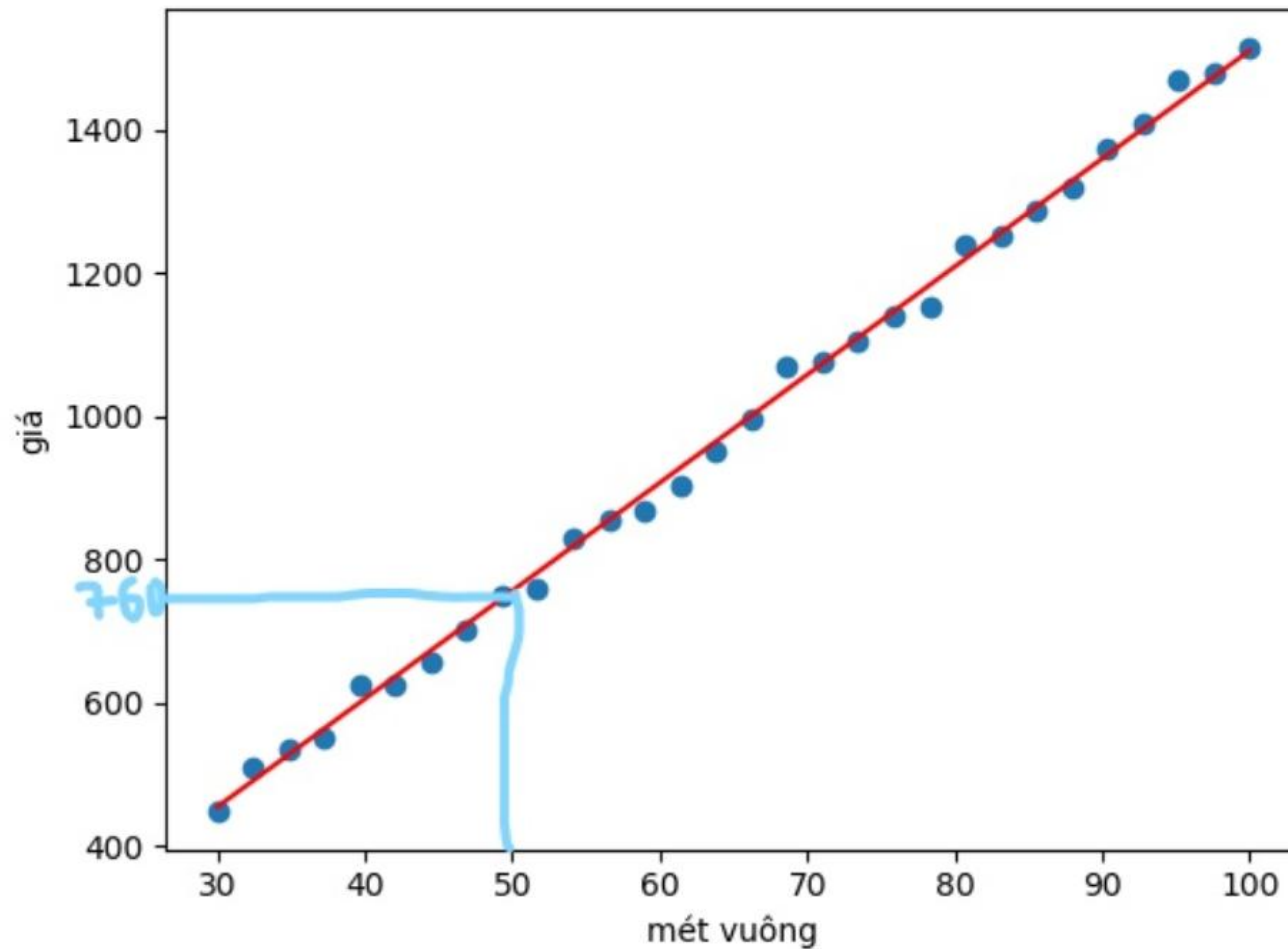


Quan hệ giữa diện tích và giá nhà

Linear regression (Hồi quy tuyến tính)

- **Bài toán:**

- Nếu giờ yêu cầu bạn ước lượng nhà 50 mét vuông khoảng bao nhiêu tiền thì bạn sẽ làm thế nào? Vẽ một đường thẳng gần với các điểm trên nhất và tính giá nhà ở điểm 50.



Ước tính căn nhà 50 m²

Linear regression (Hồi quy tuyến tính)

- Về mặt lập trình cũng cần làm 2 việc như vậy:
 - **Huấn luyện (Training):** Tìm đường thẳng (model) gần các điểm trên nhất. Mọi người có thể vẽ ngay được đường thẳng mô tả dữ liệu từ hình ở slide trước, nhưng máy tính thì không, nó phải đi tìm bằng thuật toán **Gradient descent**.
 - **Prediction:** Dự đoán xem giá của ngôi nhà 50 m² có giá bao nhiêu dựa trên đường tìm được ở phần trên.

Linear regression (Hồi quy tuyến tính)

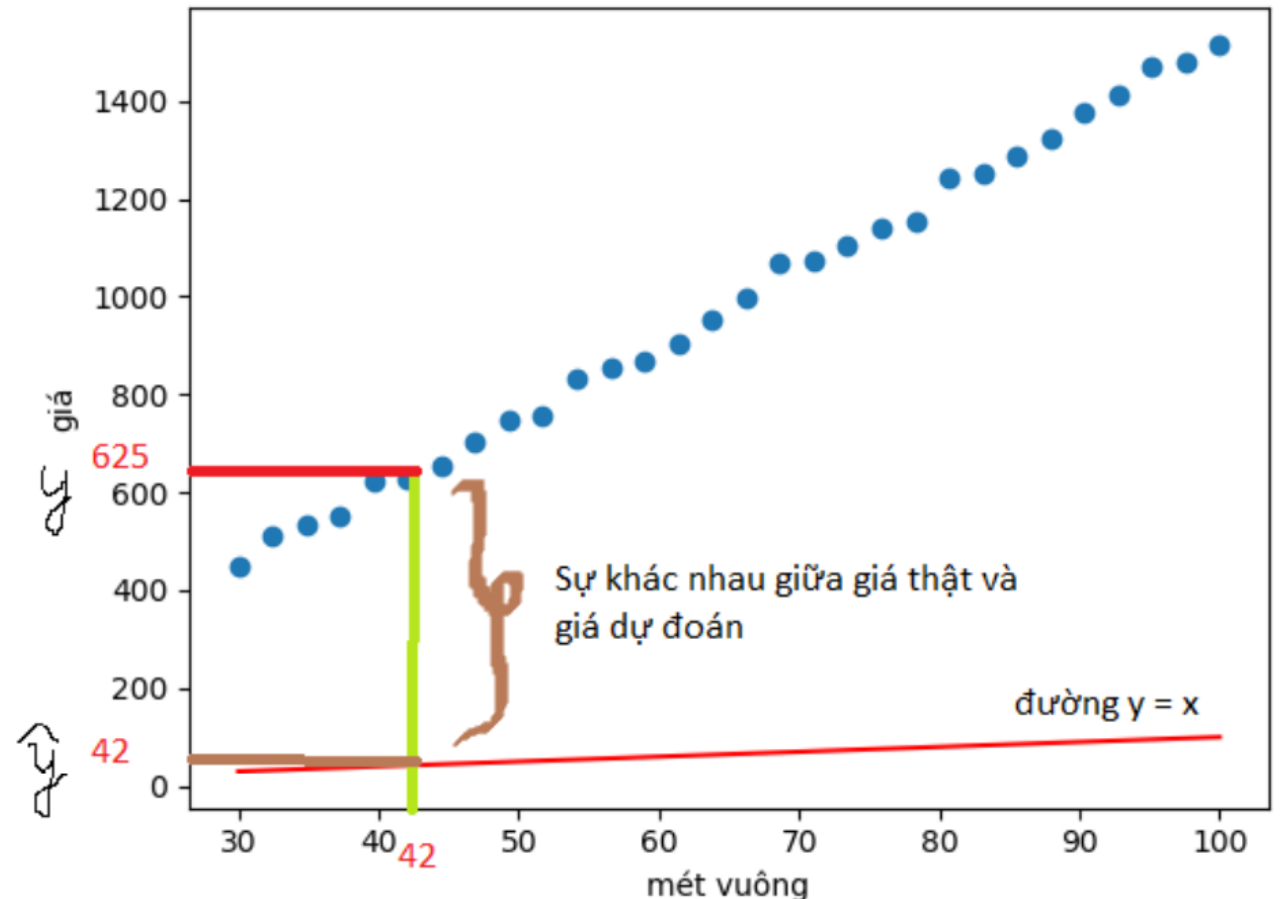
- Thiết lập công thức:
 - **Mô hình (Model):** Với phương trình đường thẳng có dạng $y = ax + b$, để tiện cho biểu diễn ma trận phần sau ta sẽ thay $w_1 = a$ và $w_0 = b$
 - Phương trình được viết lại thành: $y = w_1 * x + w_0 \Rightarrow$ Việc tìm đường thẳng giờ thành việc tìm w_0 và w_1 .
 - Để tiện cho việc thiết lập công thức, ta sẽ đặt ký hiệu cho dữ liệu ở bảng dữ liệu: $(x_1, y_1) = (30, 448.524)$, $(x_2, y_2) = (32.4138, 509.248)$, ...
 - Tức là nhà diện tích x_i thực sự có giá y_i . Còn giá trị mà model hiện tại đang dự đoán kí hiệu là $\hat{y}_i = w_1 * x_i + w_0$

Linear regression (Hồi quy tuyến tính)

- Thiết lập công thức:
 - Loss function:** Việc tìm w_0 và w_1 có thể đơn giản nếu làm thủ công nhưng máy tính không biết điều đấy, nên ban đầu giá trị được chọn ngẫu nhiên ví dụ $w_0 = 0$ và $w_1 = 1$ sau đấy được chỉnh dần.

Đường $y = x$ như ở hình bên không hề gần các điểm hay không phải là đường mà ta cần tìm.

Ví dụ tại điểm $x = 42$ (nhà 42 m^2) giá thật là 625 triệu nhưng giá mà model dự đoán chỉ là 42 triệu.



Linear regression (Hồi quy tuyến tính)

- Thiết lập công thức:
 - Giờ cần một hàm để đánh giá là đường thẳng với bộ tham số $(w_0, w_1) = (0, 1)$ có tốt hay không.
 - Với mỗi điểm dữ liệu (x_i, y_i) , độ chênh lệch giữa giá thật và giá dự đoán được tính bằng: $\frac{1}{2} * (\hat{y}_i - y_i)^2$
 - Độ chênh lệch trên toàn bộ dữ liệu tính bằng tổng chênh lệch của từng điểm:

$$J = \frac{1}{2} * \frac{1}{N} * \left(\sum_{i=1}^N (\hat{y}_i - y_i)^2 \right)$$

- J không âm
- J càng nhỏ thì đường thẳng càng gần điểm dữ liệu.
- Nếu $J = 0$ thì đường thẳng đi qua tất cả các điểm dữ liệu.

Linear regression (Hồi quy tuyến tính)

- Thiết lập công thức:
 - J được gọi là loss function (hàm mất mát), hàm để đánh giá xem bộ tham số hiện tại có tốt với dữ liệu không.

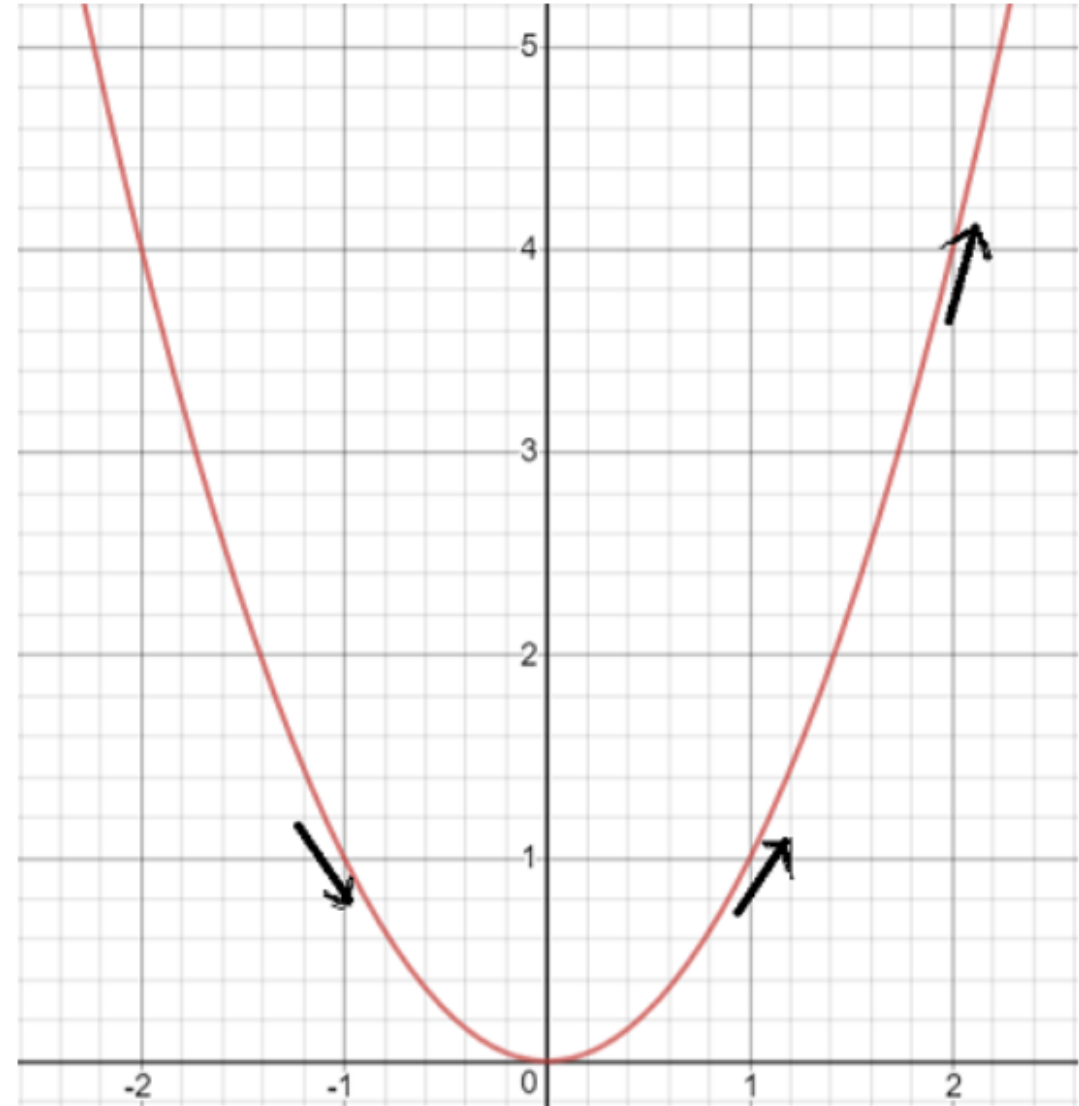
=> Bài toán tìm đường thẳng gần các điểm dữ liệu nhất, do đó trở thành tìm w_0 và w_1 sao cho hàm J đạt giá trị nhỏ nhất.
- **Tóm lại:** Cần tìm đường thẳng (model) khớp (fit) nhất với dữ liệu, tương ứng với việc tìm các tham số w_0 và w_1 để cực tiểu hóa hàm J.
- Giờ cần một thuật toán để tìm giá trị nhỏ nhất của hàm $J(w_0, w_1)$. Đó chính là thuật toán **gradient descent**.

Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent
 - Đạo hàm

Đạo hàm $f(x) = x^2$ là $f'(x) = \frac{df(x)}{dx} = 2 * x$

- $f'(1) = 2 * 1 < f'(2) = 2 * 2$ nên có thể thấy trên hình là đồ thị tại điểm $x = 2$ dốc hơn đồ thị tại điểm $x = 1$, tuy nhiên $f'(-2) = -4 < f'(-1) = -2$ nhưng đồ thị tại $x = -2$ dốc hơn đồ thị tại $x = -1$ => trị tuyệt đối của đạo hàm tại một điểm càng lớn thì đồ thị tại điểm đấy càng dốc.
- $f'(-1) = 2 * (-1) = -2 < 0$ => đồ thị đang giảm hay khi tăng x thì y sẽ giảm; ngược lại đạo hàm tại điểm nào đó mà dương thì đồ thị tại điểm đấy đang tăng.



Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent
 - là thuật toán tìm giá trị nhỏ nhất của hàm số $f(x)$ dựa trên đạo hàm.
 - Thuật toán:
 1. Khởi tạo giá trị $x = x_0$ tùy ý
 2. Gán $x = x - \text{learning_rate} * f'(x)$, learning_rate là hằng số dương. Ví dụ learning_rate = 0.001
 3. Tính lại $f(x)$: Nếu $f(x)$ đủ nhỏ thì dừng lại, ngược lại tiếp tục bước 2

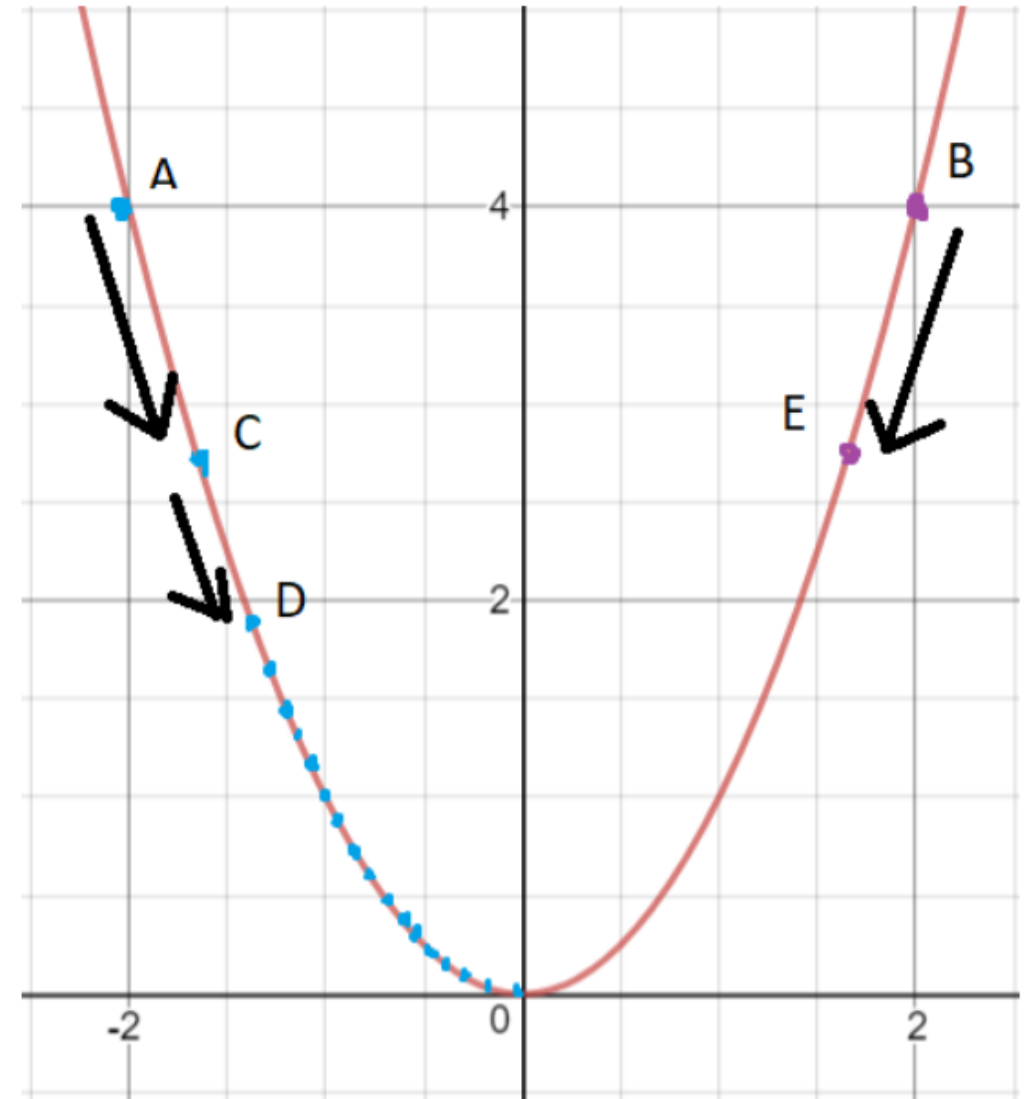
Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent

- Bước 1: Khởi tạo giá trị ngẫu nhiên $x = -2$ (điểm A).
- Bước 2: Do ở A đồ thị giảm nên $f'(x=-2) = 2*(-2) = -4 < 0 \Rightarrow$ Khi gán $x = x - \text{learning_rate} * f'(x)$ nên khi x tăng thì đồ thị bước tiếp theo ở điểm C.

Tiếp tục thực hiện Bước 2, gán $x = x - \text{learning_rate} * f'(x)$ thì đồ thị ở điểm D, ...
 $\Rightarrow$ hàm số giảm dần dần tiến tới giá trị nhỏ nhất.

Cần để ý là trị tuyệt đối của đạo hàm tại A lớn hơn tại C và tại C lớn hơn tại D. Đến khi đến gần điểm đạt giá trị nhỏ nhất $x = 0$ thì đạo hàm xấp xỉ 0, đến khi hàm đạt giá trị nhỏ nhất tại $x = 0$ thì đạo hàm bằng 0 nên tại điểm gần giá trị nhỏ nhất thì bước 2 gán $x = x - \text{learning_rate} * f'(x)$ là không đáng kể và gần như là giữ nguyên giá trị của x .

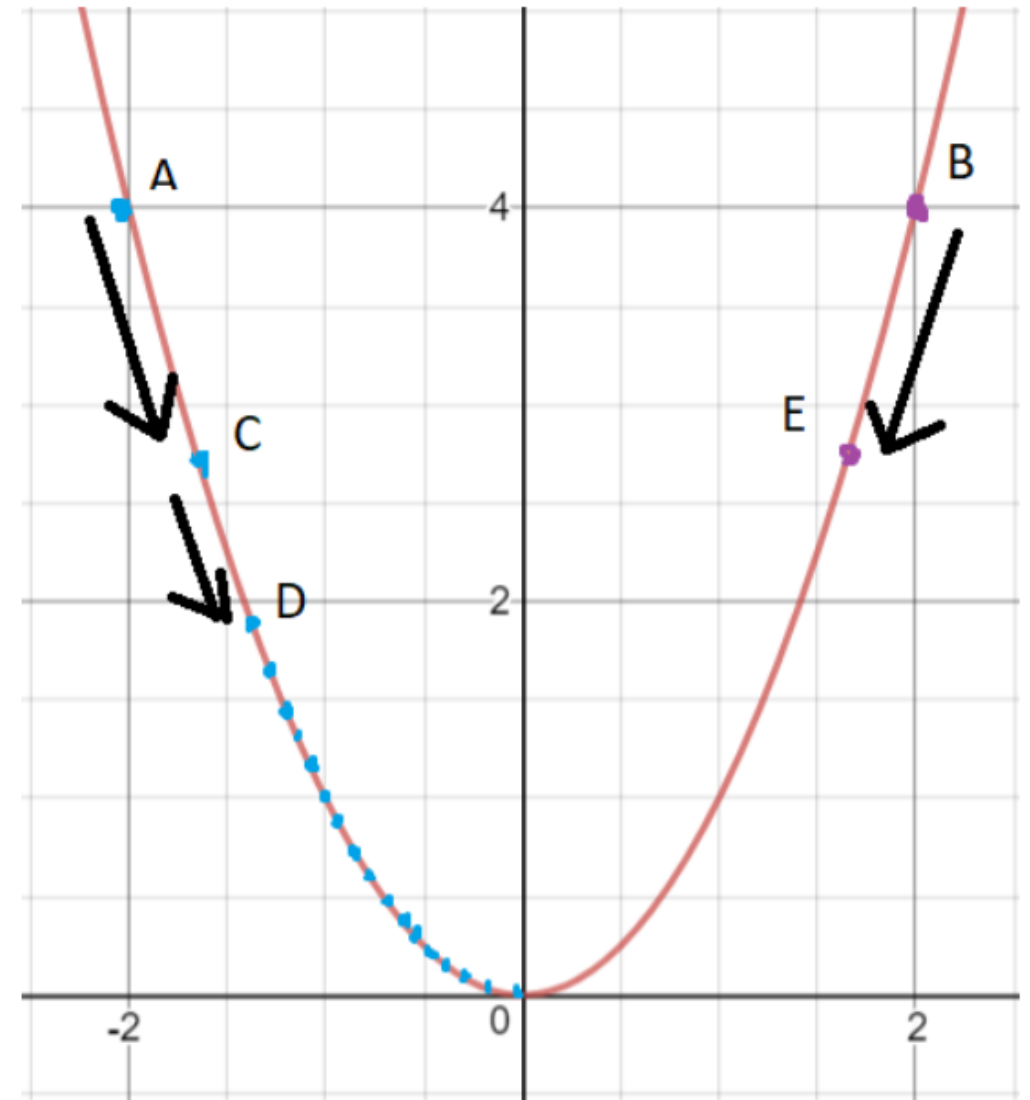


Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent

Tương tự nếu giá trị khởi tạo tại $x = 2$ (tại B) thì đạo hàm tại B dương nên do $x = x - \text{learning_rate} * f'(x)$ giảm $\rightarrow$ đồ thị bước đến điểm E.

Tiếp tục gán $x = x - \text{learning_rate} * f'(x)$ thì hàm $f(x)$ cũng sẽ giảm dần dần đến giá trị nhỏ nhất.



Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent

- Ví dụ:

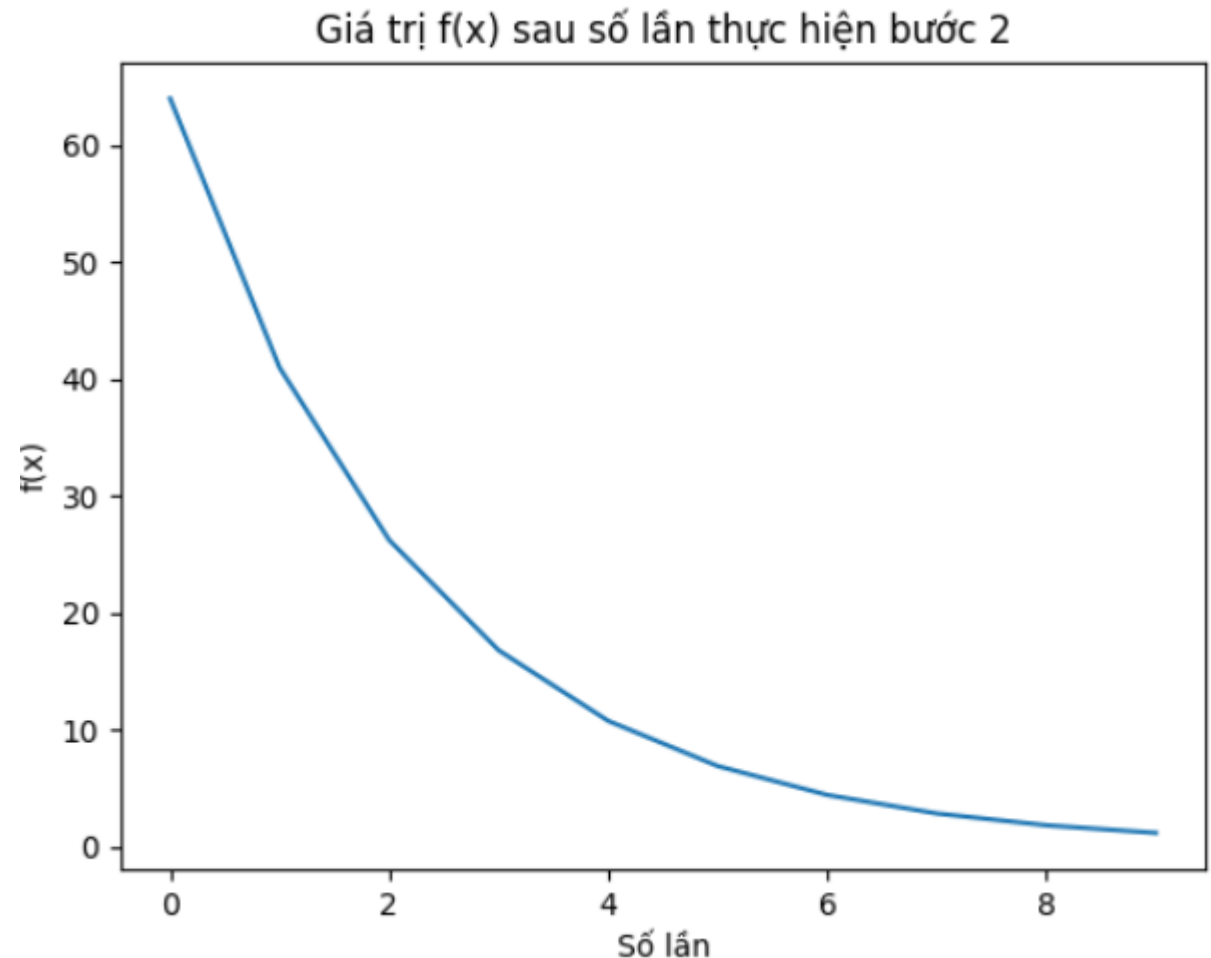
Chọn $x = 10$, $\text{learning_rate} = 0.1$, bước 2 sẽ cập nhật $x = x - \text{learning_rate} * f'(x) = x - \text{learning_rate} * 2 * x$, giá trị $f(x) = x^2$ sẽ thay đổi qua các lần thực hiện bước 2 như bảng bên:

Lần	x	f(x)
1	8.00	64.00
2	6.40	40.96
3	5.12	26.21
4	4.10	16.78
5	3.28	10.74
6	2.62	6.87
7	2.10	4.40
8	1.68	2.81
9	1.34	1.80
10	1.07	1.15

Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent

```
import numpy as np
import matplotlib.pyplot as plt
x = 10
y = []
for i in range(10):
    x = x - 0.1 * 2 * x
    y.append(x**2)
plt.plot(y)
plt.xlabel('Số lần')
plt.ylabel('f(x)')
plt.title('Giá trị f(x) sau số lần thực hiện bước 2')
plt.show()
```

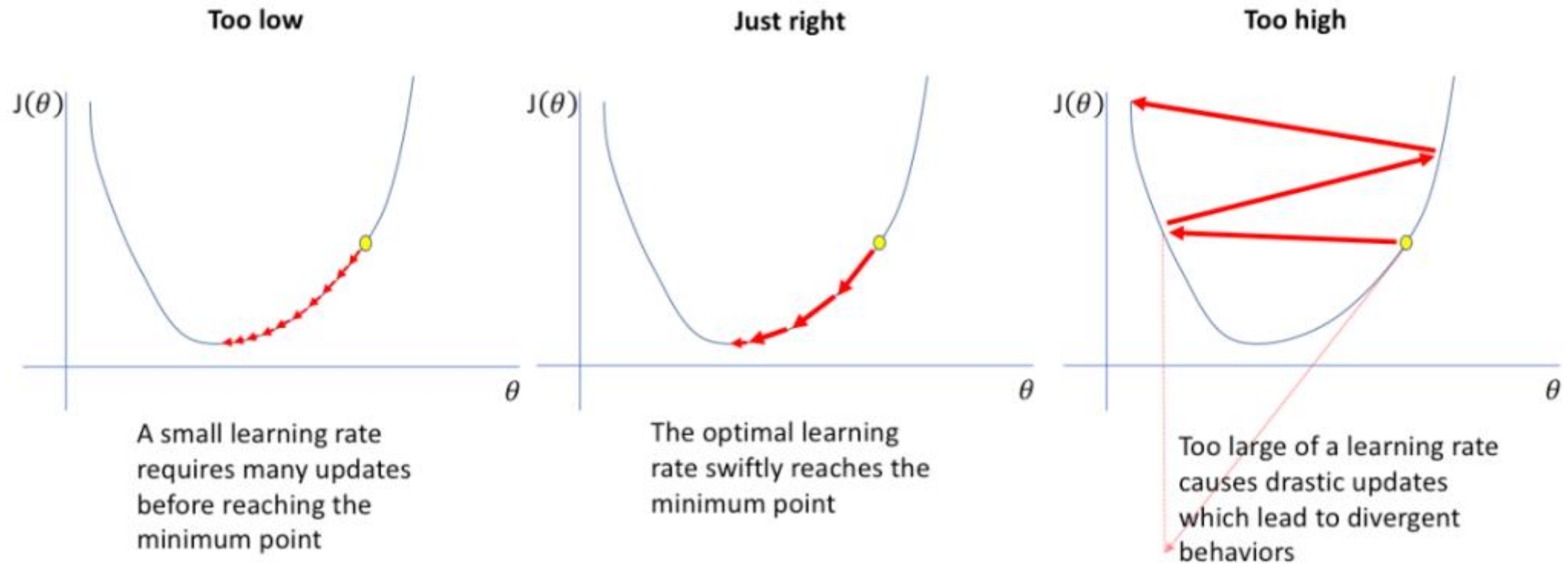


Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent
 - Nhận xét:
 - Thuật toán hoạt động rất tốt trong trường hợp không thể tìm giá trị nhỏ nhất bằng đại số tuyến tính.
 - Việc quan trọng nhất của thuật toán là tính đạo hàm của hàm số theo từng biến sau đó lặp lại bước 2.
 - Việc chọn hệ số `learning_rate` cực kì quan trọng, có 3 trường hợp:
 - Nếu `learning_rate` nhỏ: mỗi lần hàm số giảm rất ít nên cần rất nhiều lần thực hiện bước 2 để hàm số đạt giá trị nhỏ nhất.
 - Nếu `learning_rate` hợp lý: sau một số lần lặp bước 2 vừa phải thì hàm sẽ đạt giá trị đủ nhỏ.
 - Nếu `learning_rate` quá lớn: sẽ gây hiện tượng overshoot (như trong hình ở slide tiếp theo) và không bao giờ đạt được giá trị nhỏ nhất của hàm.

Linear regression (Hồi quy tuyến tính)

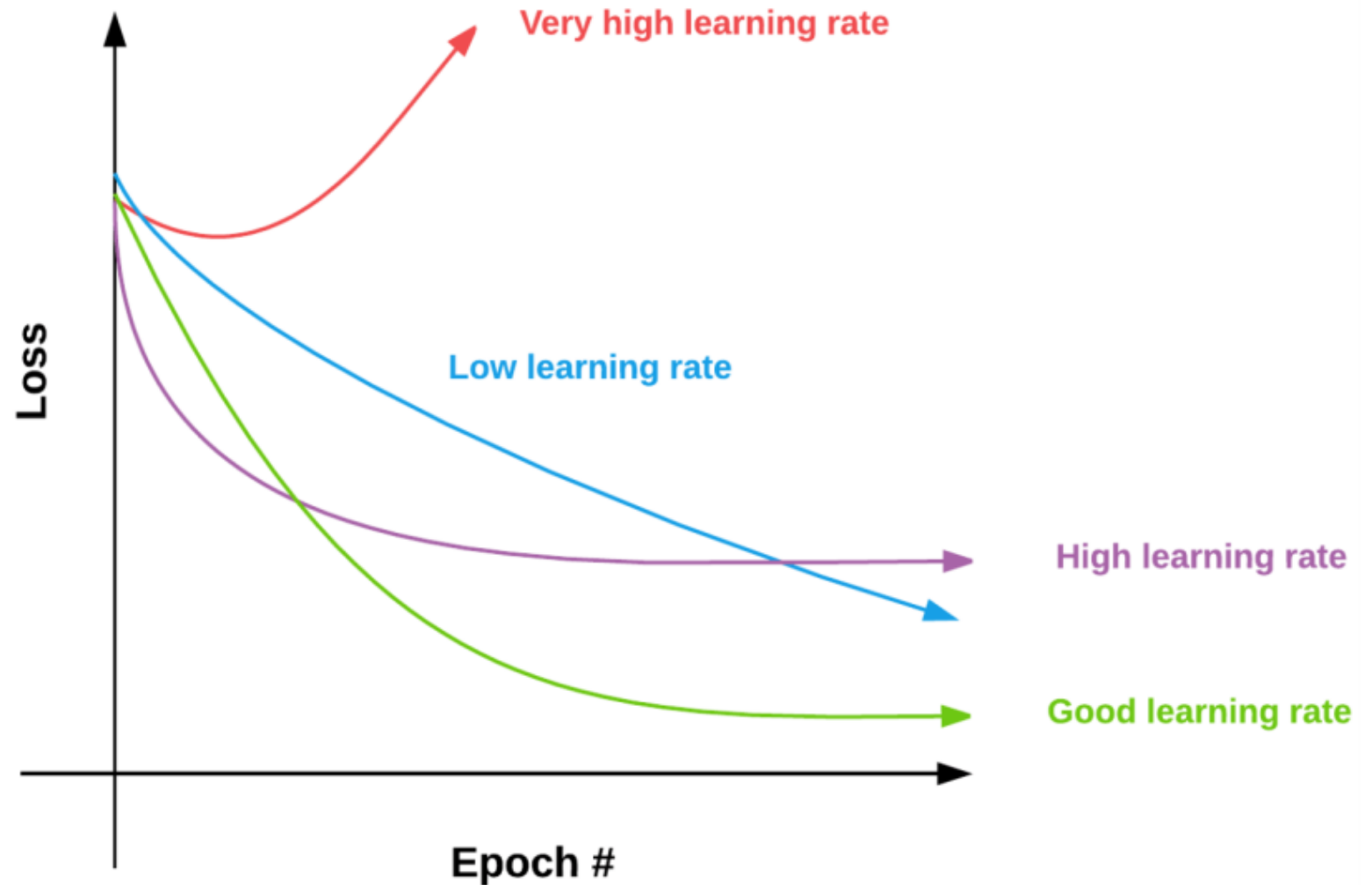
- Thuật toán gradient descent



Linear regression (Hồi quy tuyến tính)

- Thuật toán gradient descent

Cách tốt nhất để kiểm tra learning_rate hợp lý hay không là kiểm tra giá trị hàm $f(x)$ sau mỗi lần thực hiện bước 2 bằng cách vẽ đồ thị với trục x là số lần thực hiện bước 2, trục y là giá trị loss function tương ứng ở bước đấy.



Ma trận (matrix)

- Ma trận là gì
 - Ma trận là một mảng chữ nhật có m hàng và n cột, ta gọi là ma trận m * n (số hàng nhân số cột)

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

Ví dụ về một ma trận
kích thước 3*4

$$\begin{bmatrix} 2 & -5 & -11 & 0 \\ 4 & 2 & 1 & 4 \\ -1 & 3 & 0 & 5 \end{bmatrix}$$

- Chỉ số ma trận thì hàng trước cột sau ví dụ $A[1, 2] = -5$ (hàng 1, cột 2); $A[2, 1] = 4$ (hàng 2, cột 1)

Ma trận (matrix)

- Phép nhân ma trận

- Phép tính nhân ma trận $A * B$ chỉ thực hiện được khi số cột của A bằng số hàng của B , hay A có kích thước $m*n$ và B có kích thước $n*k$.
- Ma trận $C = A * B$ thì C có kích thước $m * k$ và $C[i, j] = \sum_{k=1}^n A[i, k] * B[k, j]$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

- Ví dụ: $C[1][1] = A[1][1] * B[1][1] + A[1][2] * B[2][1] = ax + cy$

$$C[2][1] = A[2][1] * B[1][1] + A[2][2] * B[2][1] = bx + dy$$

Ma trận (matrix)

- Element-wise multiplication matrix
 - Ma trận A và B cùng kích thước $m \times n$ thì phép tính này cho ra ma trận C cùng kích thước $m \times n$ và $C[i,j] = A[i,j] * B[i,j]$. Hay là mỗi phần tử ở ma trận C bằng tích 2 phần tử tương ứng ở A và B. Kí hiệu $C = A \otimes B$, ví dụ:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \otimes \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} * b_{11} & a_{12} * b_{12} \\ a_{21} * b_{21} & a_{22} * b_{22} \end{bmatrix}$$

Ma trận (matrix)

- Biểu diễn bài toán
 - Do với mỗi điểm x_i , y_i ta cần phải tính $(w_0 + w_1 * x_i - y_i)$ nên thay vì tính cho từng điểm dữ liệu, ta sẽ biểu diễn dưới dạng ma trận ma trận X kích thước $n * 2$ và Y kích thước $n * 1$ (n là số điểm dữ liệu trong tập dữ liệu mà ta có).

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \dots & \dots \\ 1 & x_n \end{bmatrix}, y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}, w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$
$$\hat{y} = X * W = \begin{bmatrix} w_0 + w_1 * x_1 \\ w_0 + w_1 * x_2 \\ \dots \\ w_0 + w_1 * x_n \end{bmatrix}$$

Ma trận (matrix)

- Biểu diễn bài toán

- $X[:, i]$ hiểu là ma trận kích thước $n \times 1$ lấy dữ liệu từ cột thứ i của ma trận X , nhưng do trong python chỉ số bắt đầu từ 0, nên cột đầu tiên là cột 0, cột thứ hai là cột 1,
Phép tính $\text{sum}(X)$ là tính tổng tất cả các phần tử trong ma trận X .

$$X[:, 1] = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$$

$$\text{sum}(X[:, 1]) = x_1 + x_2 + \dots + x_n$$

$$\frac{dJ}{dw_0} = \text{sum}(\hat{y} - y)$$

$$\frac{dJ}{dw_1} = \text{sum}(X[:, 1] \otimes (\hat{y} - y))$$

$$\frac{dJ}{dw} = X^T * (\hat{y} - y)$$

Biểu diễn đạo hàm dạng
ma trận

So sánh các loss function

- Mean Absolute Error, L1 Loss
 - Mean Absolute Error (MAE) hay còn được gọi là L1 Loss là một loss function được sử dụng cho các mô hình hồi quy, đặc biệt cho các mô hình hồi quy tuyến tính.
 - MAE được tính bằng tổng các trị tuyệt đối của hiệu giữa giá trị thực (y_i : target) và giá trị mà mô hình của chúng ta dự đoán ($\hat{y}_i$: predicted).

$$MAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

So sánh các loss function

- Mean Square Error, L2 Loss
 - Mean Square Error (MSE) hay còn được gọi là L2 Loss là một loss function cũng được sử dụng cho các mô hình hồi quy, đặc biệt là các mô hình hồi quy tuyến tính.
 - MSE được tính bằng tổng các bình phương của hiệu giữa giá trị thực (y_i : target) và giá trị mà mô hình của chúng ra dự đoán ($\hat{y}_i$: predicted).

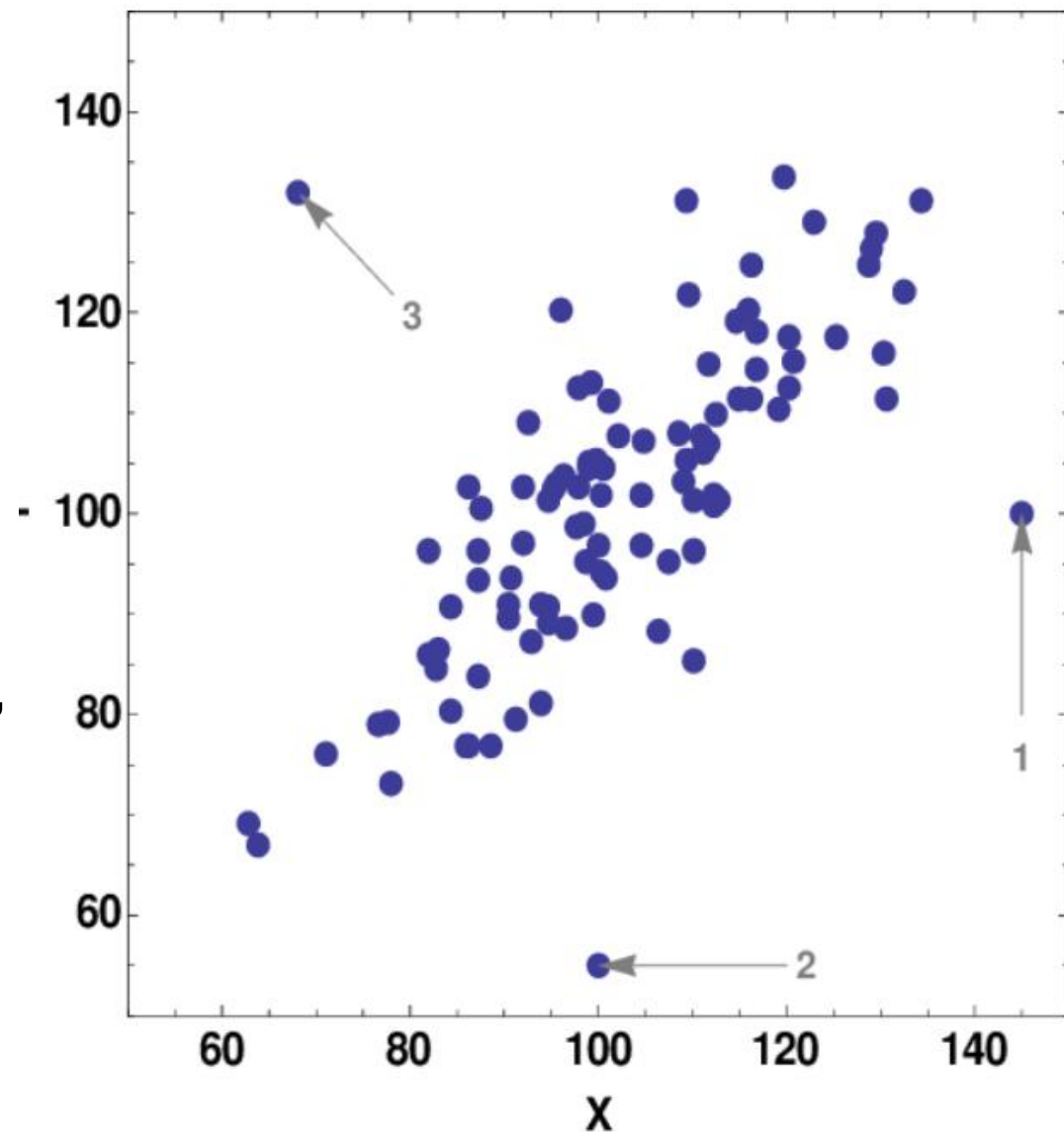
$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

So sánh các loss function

- So sánh MAE và MSE (L1 Loss và L2 Loss)
 - Để tìm điểm cực tiểu của các hàm số, thông thường cách đơn giản nhất chúng ta nghĩ đến chính là tìm đạo hàm của hàm số rồi tìm điểm mà tại đó đạo hàm của hàm số bằng 0.
 - Như vậy sẽ có 1 bước đạo hàm, hiển nhiên tìm đạo hàm của MSE sẽ đơn giản hơn rất nhiều so với MAE.
 - MAE thì đem lại kết quả tốt hơn đối với các dữ liệu có outlier (điểm **dị biệt** hay điểm **ngoại lai** - là những điểm có giá trị khác xa so với phần còn lại của dữ liệu).

So sánh các loss function

- So sánh MAE và MSE (L1 Loss và L2 Loss)
 - Điểm **ngoại lai** (dị biệt)
 - Với MSE, ta có $y_i - \hat{y}_i = e$, e^2 sẽ vô cùng lớn nếu $e > 1$.
 - Nếu bậc của e càng lớn thì giá trị hàm Loss cũng càng lớn hơn.
 - Vì vậy nếu như chúng ta có 1 outlier trong bộ dữ liệu, giá trị của e sẽ vô cùng lớn, có thể tiến tới ∞ và khi đó, e^2 có thể sẽ tiến tới ∞ , khi đó giá trị MSE cực kì lớn.



So sánh các loss function

- So sánh MAE và MSE (L1 Loss và L2 Loss)
 - Ngược lại với MSE, nếu ta có giá trị e lớn ($e > 1$) thì $|e|$ vẫn sẽ lớn, nhưng hiển nhiên nhỏ hơn nhiều so với e^2 .
 - Do đó, khi tối ưu loss function, L2 phạt mạnh hơn với các điểm outlier và model sẽ bị kéo về phía outlier hơn. Do đó, MSE bị ảnh hưởng bởi outlier và L1 tốt hơn đối với các dữ liệu có outlier.
 - Tuy vậy, đạo hàm của MAE (hay L1 Loss) tính toán khó hơn nhiều so với MSE (hay L2 Loss).

Logistic regression

- Thuật toán logistic regression giải quyết các bài toán với đầu ra là giá trị nhị phân (0 hoặc 1)
- Bài toán:
 - Ngân hàng bạn đang làm có chương trình cho vay ưu đãi cho các đối tượng mua chung cư. Tuy nhiên, gần đây có một vài chung cư rất hấp dẫn (giá tốt, vị trí đẹp, ...) nên lượng hồ sơ người nộp cho chương trình ưu đãi tăng đáng kể. Bình thường bạn có thể duyệt 10-20 hồ sơ một ngày để quyết định hồ sơ có được cho vay hay không, tuy nhiên gần đây bạn nhận được 1000-2000 hồ sơ mỗi ngày. Bạn không thể xử lý hết hồ sơ và bạn cần có một giải pháp để có thể dự đoán hồ sơ mới là có nên cho vay hay không.

Logistic regression

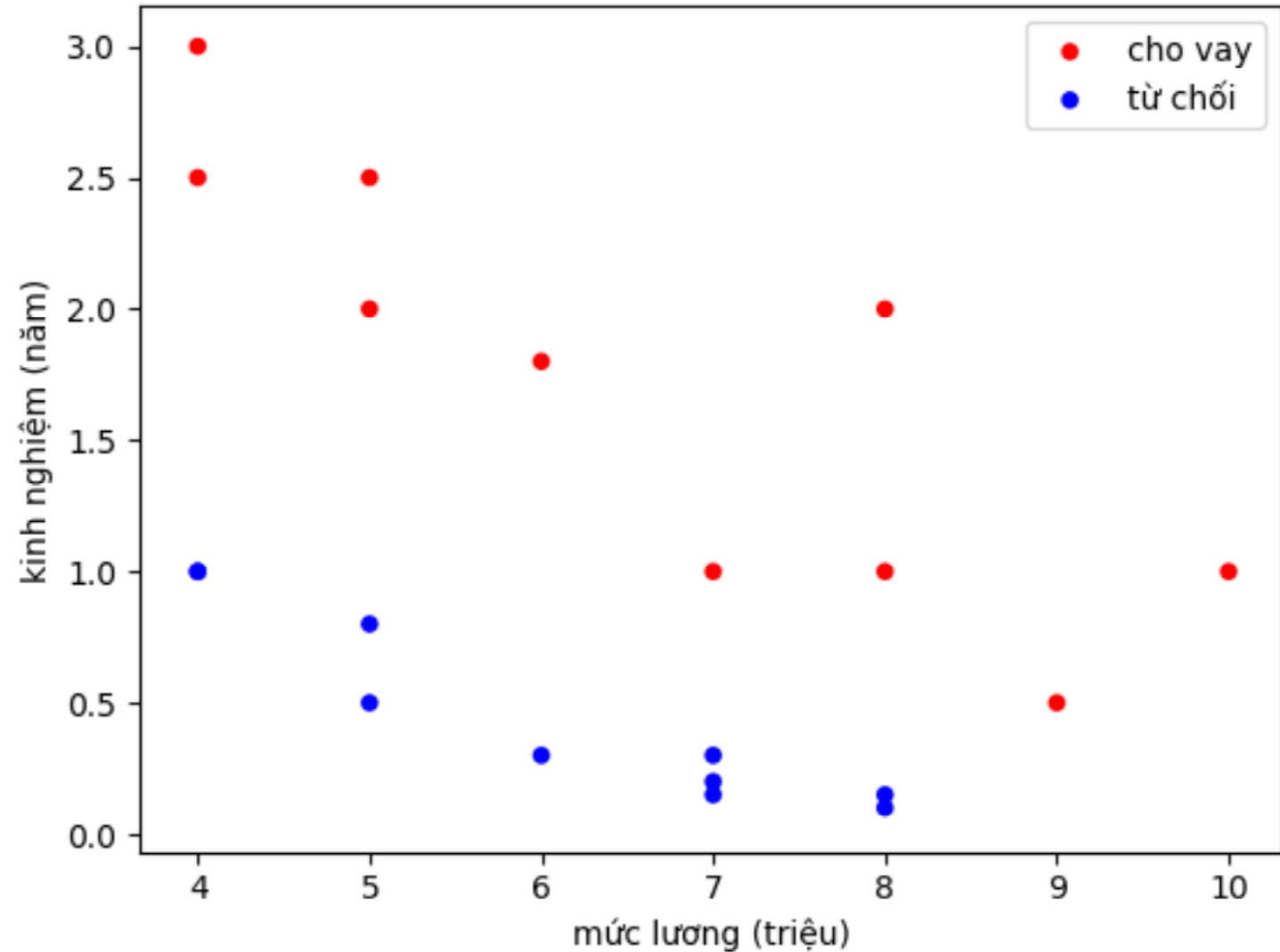
- Bài toán:
 - Sau khi phân tích thì bạn nhận thấy là hai yếu tố chính quyết định đến việc được vay tiền đó là mức lương và thời gian công tác. Đây là dữ liệu bạn có từ trước đến nay:

Lương	Thời gian làm việc	Cho vay
10	1	1
9	0.5	1
5	2	1
...	...	...
8	0.1	0
6	0.3	0
7	0.15	0
...	...	...

Logistic regression

- Bài toán:

- Trực quan hóa dữ liệu:



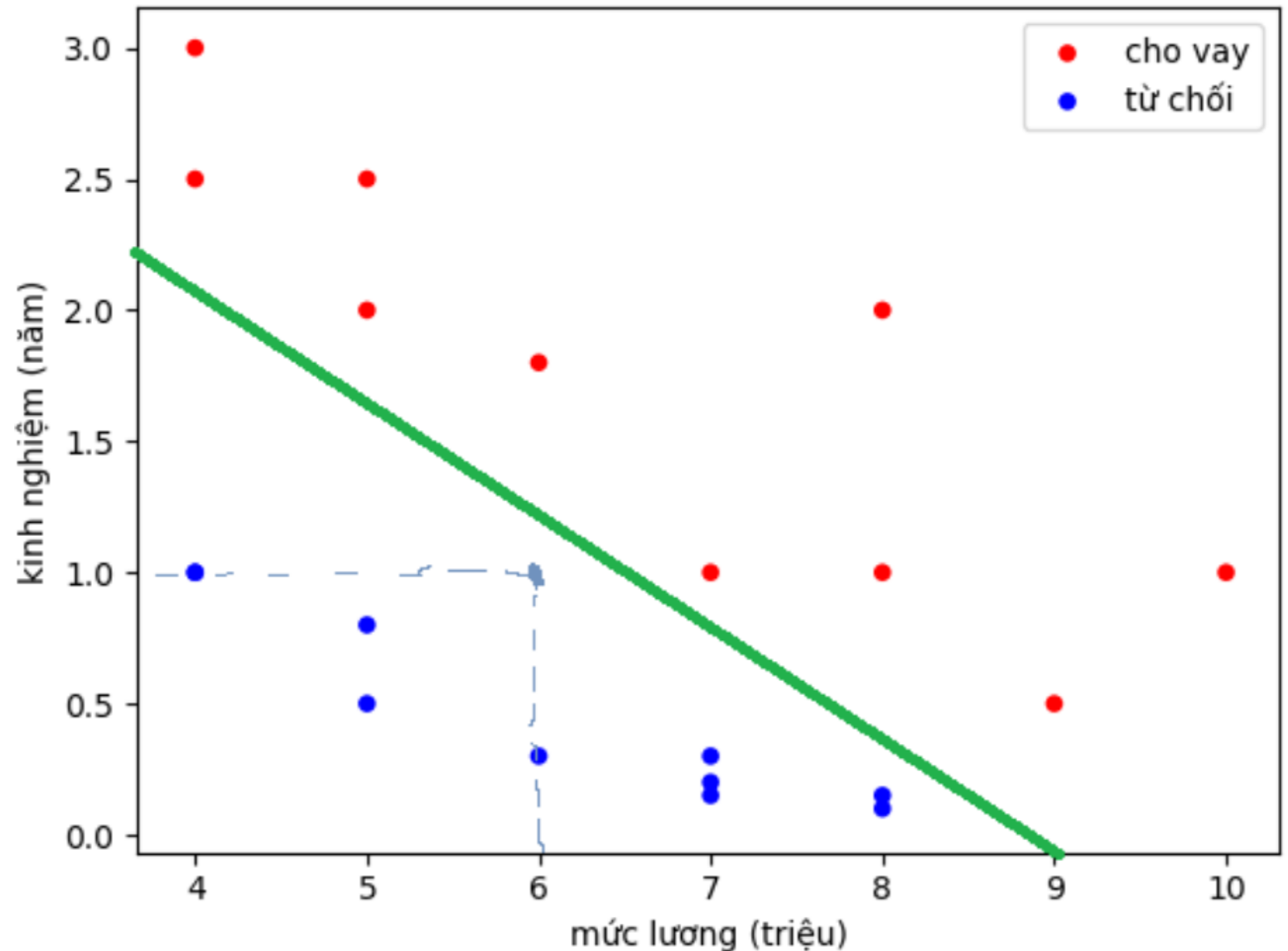
Logistic regression

- Bài toán:

Về mặt logic, ta cần tìm đường thẳng phân chia giữa các điểm cho vay và từ chối.

Sau đó quyết định hồ sơ mới có nên có vay hay không từ đường đây như hình bên.

Ví dụ đường xanh là đường phân chia. Dự đoán cho hồ sơ của người có mức lương 6 triệu và 1 năm kinh nghiệm là không cho vay.



Logistic regression

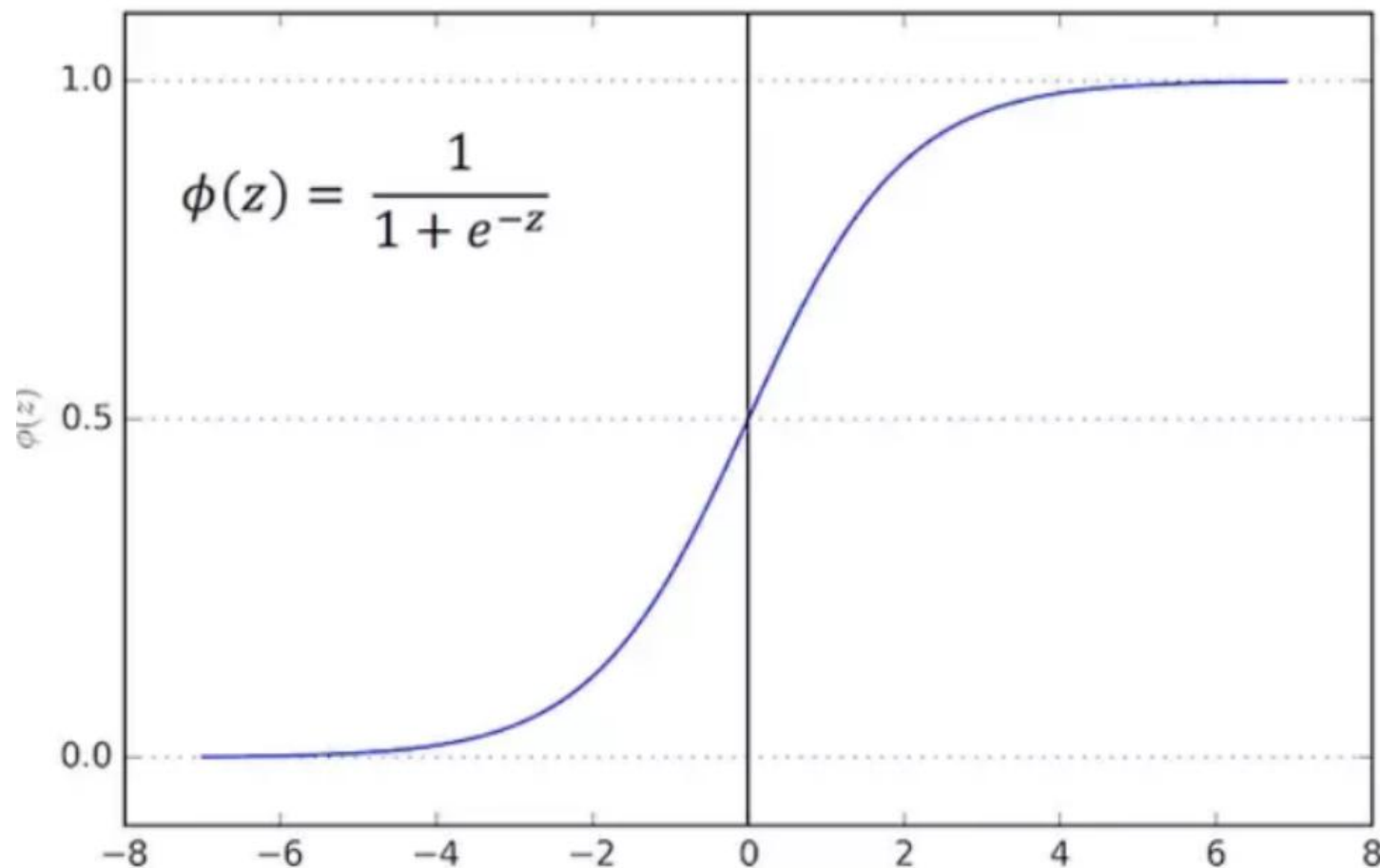
- Bài toán:
 - Tuy nhiên, do ngân hàng đang trong thời kỳ khó khăn nên việc cho vay bị thắt lại, chỉ những hồ sơ nào chắc chắn trên 80% mới được vay.
 - Vậy nên, bây giờ bạn không những tìm là hồ sơ ấy cho vay hay không cho vay mà cần tìm xác suất nên cho hồ sơ ấy vay là bao nhiêu.
 - Hay trong nhiều trường hợp khác trong bài toán phân loại người ta quan tâm hơn đến xác suất thay vì chỉ 1 hay 0. Ví dụ, bác sĩ sẽ thông báo ca mổ này 80% thành công cho người nhà bệnh nhân.

Hàm sigmoid

- Giờ ta cần tìm xác suất của hồ sơ mới nên cho vay. Hay giá trị của hàm cần trong khoảng $[0,1]$.
- Rõ ràng là giá trị của phương trình đường thẳng như bài trước có thể ra ngoài khoảng $[0,1]$ nên cần một hàm mới luôn có giá trị trong khoảng $[0,1]$. Đó là hàm sigmoid.

Hàm sigmoid

- Nhận xét
 - Hàm số liên tục, nhận giá trị thực trong khoảng (0,1).
 - Hàm có đạo hàm tại mọi điểm (để áp dụng gradient descent).



Thiết lập bài toán

- Với bài linear regression được trình bày ở trên, các bước bao gồm:
 1. Visualize dữ liệu
 2. Thiết lập model
 3. Thiết lập loss function
 4. Tìm tham số bằng việc tối ưu loss function
 5. Dự đoán dữ liệu mới bằng model vừa tìm được

➔ **Đây là mô hình chung cho bài toán trong Deep Learning.**

Thiết lập bài toán

- Mô hình:

- Với dòng thứ i trong bảng dữ liệu, gọi $x_1^{(i)}$ là lương và $x_2^{(i)}$ là thời gian làm việc của hồ sơ thứ i .
- $p(x^{(i)} = 1) = \hat{y}_i$ là xác suất mà model dự đoán hồ sơ thứ i được cho vay.
- $p(x^{(i)} = 0) = 1 - \hat{y}_i$ là xác suất mà model dự đoán hồ sơ thứ i không được cho vay.

$$\rightarrow p(x^{(i)} = 1) + p(x^{(i)} = 0) = 1$$

- Hàm sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Công thức của logistic regression là:

$$\hat{y}_i = \sigma \left(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)} \right) = \frac{1}{1 + e^{-\left(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)} \right)}}$$

Thiết lập bài toán

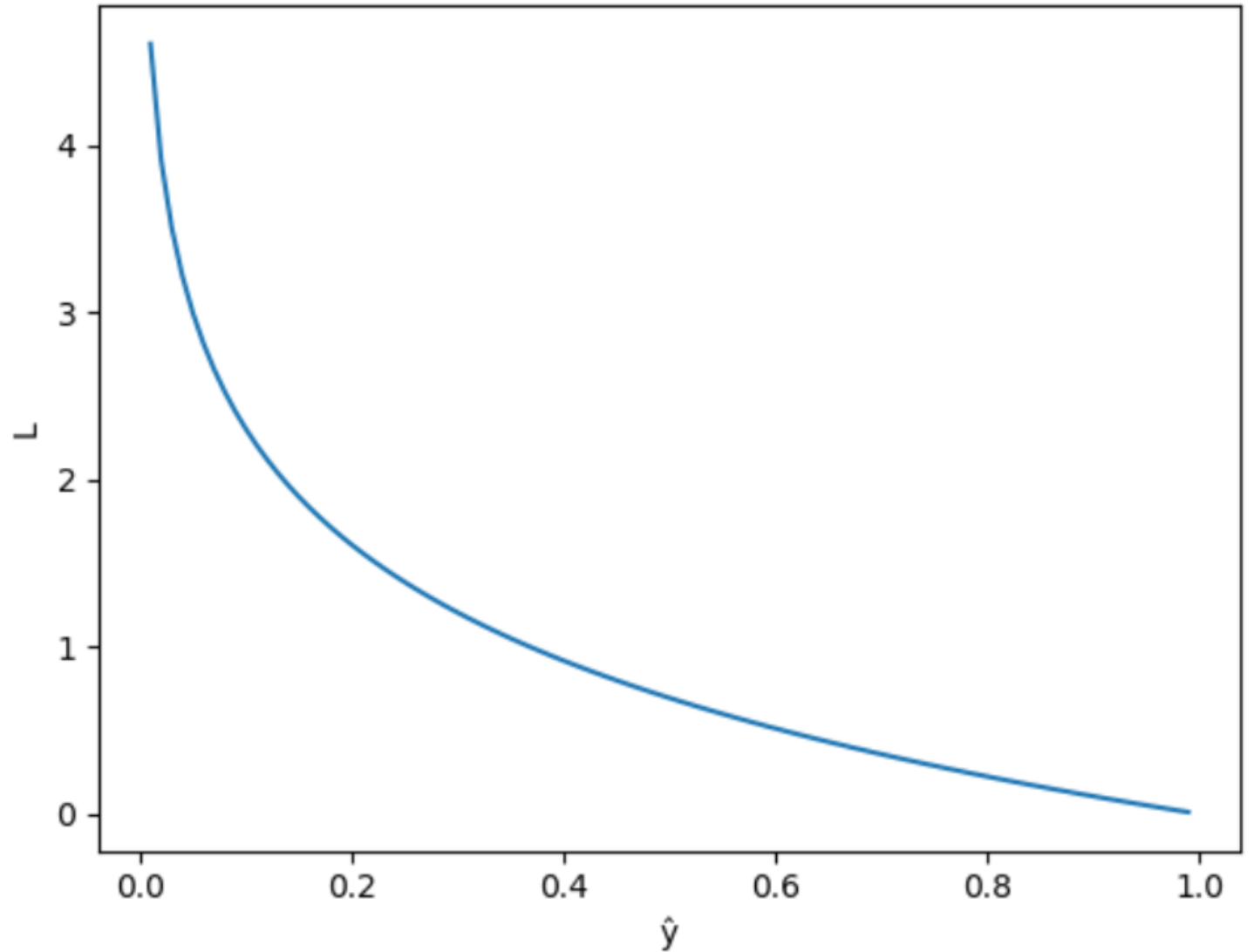
- Loss function:
 - Giờ chúng ta cần một hàm để đánh giá độ tốt của model. Như vậy, ta cần $\hat{y}_i$ càng gần y_i càng tốt.
 - Nếu hồ sơ thứ i là cho vay, tức $y_i = 1$ thì ta cũng mong muốn $\hat{y}_i$ càng gần 1 càng tốt hay model dự đoán xác suất người thứ i được vay vốn càng cao càng tốt.
 - Nếu hồ sơ thứ i không được vay, tức $y_i = 0$ thì ta cũng mong muốn $\hat{y}_i$ càng gần 0 càng tốt hay model dự đoán xác suất người thứ i được vay vốn càng thấp càng tốt.
 - Với mỗi điểm $(x^{(i)}, y_i)$, gọi hàm loss function:
$$L = -(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$$
loss function này có tên gọi là binary_crossentropy

Mặc định trong machine learning nói chung hay deep learning thì viết log hiểu là ln

Thiết lập bài toán

- Loss function:

- Nếu $y_i = 1 \Rightarrow L = -\log(\hat{y}_i)$
- Nhận xét:
 - Hàm L giảm dần từ 0 đến 1.
 - Khi model dự đoán $\hat{y}_i$ gần 1, tức giá trị dự đoán gần với giá trị thật y_i thì L nhỏ, xấp xỉ 0
 - Khi model dự đoán $\hat{y}_i$ gần 0, tức giá trị dự đoán ngược lại giá trị thật y_i thì L rất lớn



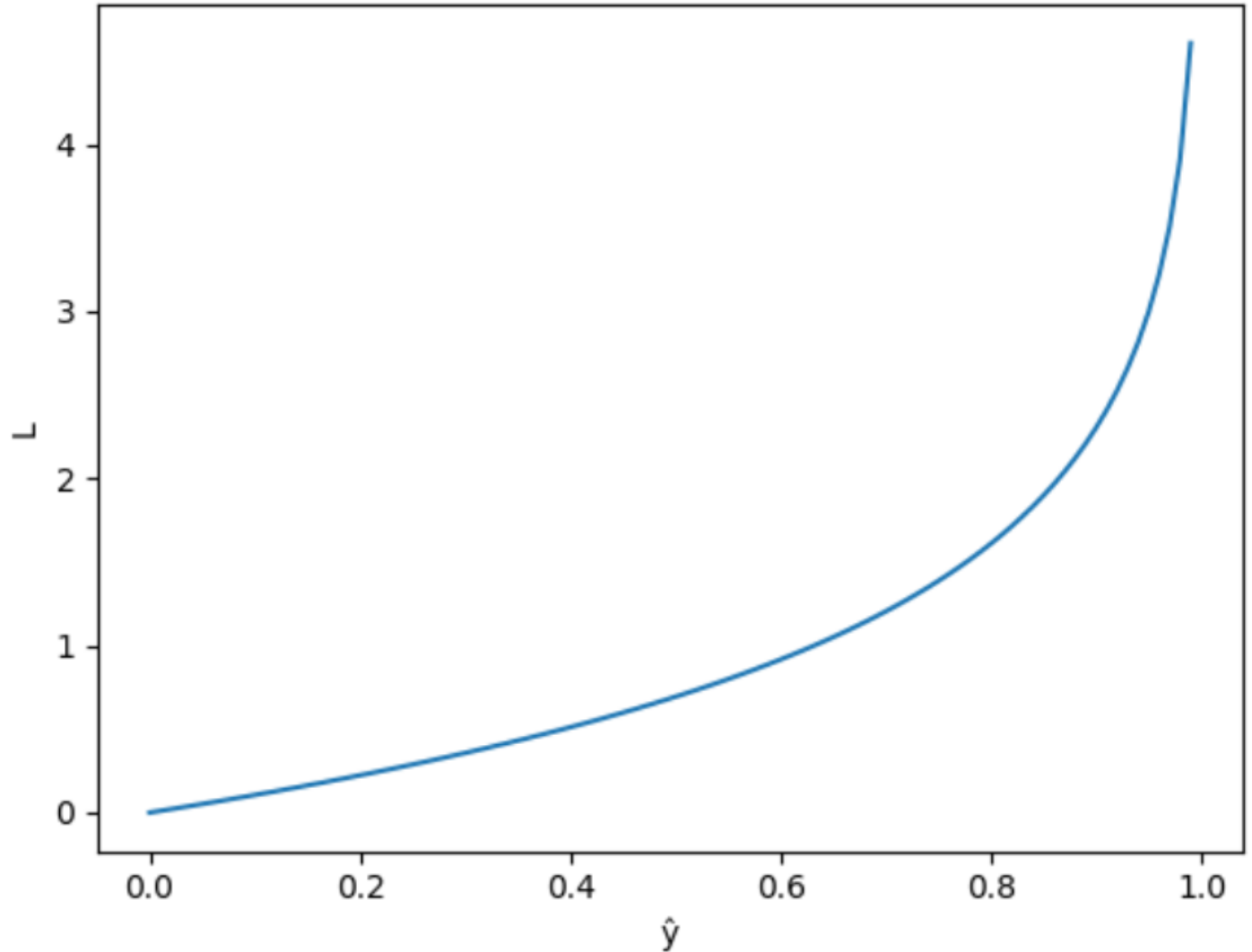
Thiết lập bài toán

- Loss function:

- Nếu $y_i = 0 \Rightarrow L = -\log(1 - \hat{y}_i)$

- Nhận xét:

- Hàm L tăng dần từ 0 đến 1.
- Khi model dự đoán $\hat{y}_i$ gần 0, tức giá trị dự đoán gần với giá trị thật y_i thì L nhỏ, xấp xỉ 0
- Khi model dự đoán $\hat{y}_i$ gần 1, tức giá trị dự đoán ngược lại giá trị thật y_i thì L rất lớn



Thiết lập bài toán

- Loss function:
 - **Kết luận:** Hàm L nhỏ khi giá trị model dự đoán gần với giá trị thật và rất lớn khi model dự đoán sai. Nói cách khác L càng nhỏ thì model dự đoán càng gần với giá trị thật.

=> **Bài toán tìm model trở thành tìm giá trị nhỏ nhất của L.**

- Hàm loss function trên toàn bộ dữ liệu:

$$J = -\frac{1}{N} * \sum_{i=1}^N (y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$$

Chain rule

- Chain rule là gì?

- Nếu $z = f(y)$ và $y = g(x)$ hay $z = f(g(x))$ thì $\frac{dz}{dx} = \frac{dz}{dy} * \frac{dy}{dx}$

- Ví dụ, cần tính đạo hàm $z(x) = (2x + 1)^2$,
có thể thấy $z = f(g(x))$ trong đó $f(x) = x^2, g(x) = 2x + 1$.

- Do đó, áp dụng chain rule ta có:

$$\frac{dz}{dx} = \frac{dz}{dy} * \frac{dy}{dx} = \frac{d(2x+1)^2}{d(2x+1)} * \frac{d(2x+1)}{dx} = 2 * (2x+1) * 2 = 4 * (2x+1)$$

$$\frac{dt^2}{dt} = 2t \Rightarrow \frac{d(2x+1)^2}{d(2x+1)} = 2(2x+1) \text{ bằng cách đặt } t = 2x+1.$$

Chain rule

- Áp dụng tính đạo hàm của hàm sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$
 - Nhắc lại kiến thức đạo hàm cơ bản: $\frac{d(\frac{1}{x})}{dx} = \frac{-1}{x^2}$, $\frac{d(e^x)}{dx} = e^x$.
 - Do đó:

$$\begin{aligned}\frac{d(\sigma(x))}{dx} &= \frac{d(\frac{1}{1+e^{-x}})}{dx} = \frac{-1}{(1+e^{-x})^2} * 1 * e^{-x} * (-1) = \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} * \frac{e^{-x}}{1+e^{-x}} = \frac{1}{1+e^{-x}} * (1 - \frac{1}{1+e^{-x}}) = \sigma(x) * (1 - \sigma(x))\end{aligned}$$

Áp dụng gradient descent

- Để áp dụng thuật toán gradient descent tìm tối ưu loss function cần tính đạo hàm của loss function với w .

- Với mỗi điểm $(x^{(i)}, y_i)$, gọi hàm loss function:

$L = -(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))$, trong đó

$\hat{y}_i = \sigma(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)}) = \sigma(z)$ là giá trị mà model dự đoán

y_i là giá trị thật của dữ liệu, $z = w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)}$

- Với đạo hàm cơ bản $\frac{d(\log(x))}{dx} = \frac{1}{x}$

- Áp dụng chain rule ta có:

$$\frac{dL}{dw_0} = \frac{dL}{d\hat{y}_i} * \frac{d\hat{y}_i}{dz} * \frac{dz}{dw_0}$$

Áp dụng gradient descent

$$\frac{dL}{d\hat{y}_i} = -\frac{d(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))}{d\hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{(1 - \hat{y}_i)}\right) = \frac{\hat{y}_i - y_i}{\hat{y}_i * (1 - \hat{y}_i)} (*)$$

$$\frac{d\hat{y}_i}{dw_0} = \frac{d(\sigma(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)}))}{dw_0} = \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{dL}{dw_0} = \frac{dL}{d\hat{y}_i} * \frac{d\hat{y}_i}{dw_0} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{(1 - \hat{y}_i)}\right) * \hat{y}_i * (1 - \hat{y}_i) = -(y_i * (1 - \hat{y}_i) - (1 - y_i) * \hat{y}_i) = \hat{y}_i - y_i$$

$$\frac{d\hat{y}_i}{dw_1} = \frac{d(\sigma(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)}))}{dw_1} = x_1^{(i)} * \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{dL}{dw_1} = x_1^{(i)} * (\hat{y}_i - y_i)$$

$$\frac{d\hat{y}_i}{dw_2} = \frac{d(\sigma(w_0 + w_1 * x_1^{(i)} + w_2 * x_2^{(i)}))}{dw_2} = x_2^{(i)} * \hat{y}_i * (1 - \hat{y}_i)$$

$$\frac{dL}{dw_2} = x_2^{(i)} * (\hat{y}_i - y_i)$$

Áp dụng gradient descent

- Trên toàn bộ dữ liệu ta có:

$$\frac{dL}{dw_0} = \frac{1}{N} * \sum_{i=1}^N (\hat{y}_i - y_i)$$

$$\frac{dL}{dw_1} = \frac{1}{N} * \sum_{i=1}^N x_1^{(i)} * (\hat{y}_i - y_i)$$

$$\frac{dL}{dw_2} = \frac{1}{N} * \sum_{i=1}^N x_2^{(i)} * (\hat{y}_i - y_i)$$

Áp dụng gradient descent

- Biểu diễn bài toán dưới ma trận

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ \dots & \dots & \dots \\ 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix}, y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{bmatrix}, w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{y} = \sigma(X * W)$$

$$J = -\frac{1}{N} * \text{sum}(y \otimes \log(\hat{y}) + (1 - y) \otimes \log(1 - \hat{y}))$$

$$\frac{dJ}{dw} = \frac{1}{N} * X^T * (\hat{y} - y)$$

Áp dụng gradient descent

- Sau khi thực hiện thuật toán gradient descent ta sẽ tìm được w_0 , w_1 và w_2 .
- Với mỗi hồ sơ mới $x^{(t)}$ ta sẽ tính được phần trăm nên cho vay:

$$\hat{y}_t = \sigma \left(w_0 + w_1 * x_1^{(t)} + w_2 * x_2^{(t)} \right)$$

rồi so sánh với ngưỡng cho vay của công ty t (bình thường là $t = 0.5$, thời kì thiết chặt thì là $t = 0.8$) nếu $\hat{y}_t \geq t$ thì cho vay, không thì không cho vay.

Hàm sigmoid được chọn để đạo hàm đẹp

Ta có: $\frac{dL}{dw_0} = \frac{dL}{d\hat{y}_i} * \frac{d\hat{y}_i}{dz} * \frac{dz}{dw_0}$

$$\frac{dL}{d\hat{y}_i} = -\frac{d(y_i * \log(\hat{y}_i) + (1 - y_i) * \log(1 - \hat{y}_i))}{d\hat{y}_i} = -\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{(1 - \hat{y}_i)}\right) = \frac{\hat{y}_i - y_i}{\hat{y}_i * (1 - \hat{y}_i)} (*)$$

Ta có $\hat{y} = \theta(z)$ nên ta cần tìm 1 hàm $\theta(z)$ có đạo hàm $\frac{\partial \hat{y}}{\partial z}$ triệt tiêu mẫu số biểu thức (*):

Hàm sigmoid được chọn để đạo hàm đẹp

$$\Rightarrow \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

$$\Leftrightarrow \frac{\partial \hat{y}}{\hat{y}(1 - \hat{y})} = \partial z$$

$$\Leftrightarrow \int \frac{\partial \hat{y}}{\hat{y}(1 - \hat{y})} = \int \partial z$$

$$\Leftrightarrow \int \left(\frac{1}{\hat{y}} + \frac{1}{1 - \hat{y}} \right) \partial \hat{y} = \int \partial z$$

$$\Leftrightarrow \log \hat{y} - \log (1 - \hat{y}) = z$$

$$\Leftrightarrow \log \frac{\hat{y}}{1 - \hat{y}} = z$$

$$\Leftrightarrow \frac{\hat{y}}{1 - \hat{y}} = e^z$$

$$\Leftrightarrow \hat{y} = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z)$$

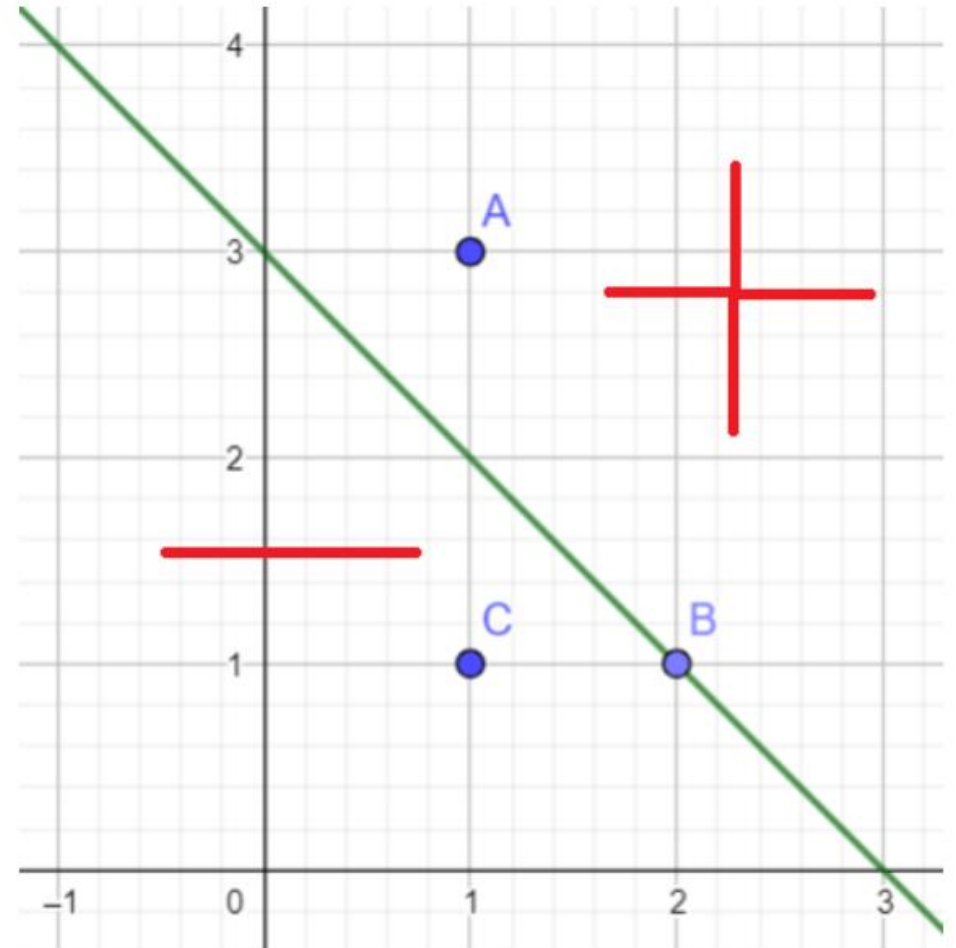
Có thể thấy hàm sigmoid ngoài việc liên tục có đạo hàm tại mọi điểm, cho giá trị trong khoảng (0, 1) thì còn làm đạo hàm loss function đẹp hơn.

Quan hệ giữa phần trăm và đường thẳng

- Xét đường thẳng $y = ax+b$, gọi $f(x,y) = y - (ax+b)$ thì đường thẳng chia mặt phẳng thành 2 miền, một miền hàm f có giá trị dương, một miền hàm f có giá trị âm và giá trị f với các điểm trên đường thẳng bằng 0.

Ví dụ, xét đường thẳng $y = 3-x$, hàm $f(x,y) = y - 3 + x$.

- Tại điểm $A(1,3) \Rightarrow f(1,3) = 3 - 3 + 1 = 1 > 0$
- Tại điểm $B(2,1)$ nằm trên đường thẳng
 $\Rightarrow f(2,1) = 2 - 3 + 1 = 0$
- Tại điểm $C(1,1) \Rightarrow f(1,1) = 1 - 3 + 1 = -1 < 0$



Quan hệ giữa phần trăm và đường thẳng

- Giả sử lấy mốc ở chính giữa là 50% tức là nếu hồ sơ mới dự đoán $\hat{y}_i \geq 0.5$ thì cho vay, còn nhỏ hơn 0.5 thì không cho vay.

$$\hat{y}_i \geq 0.5 \Leftrightarrow \frac{1}{1 + e^{-(w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)})}} \geq 0.5$$

$$\Leftrightarrow 2 \geq 1 + e^{-(w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)})}$$

$$\Leftrightarrow e^{-(w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)})} \leq 1 = e^0$$

$$\Leftrightarrow -(w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)}) \leq 0$$

$$\Leftrightarrow w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} \geq 0$$

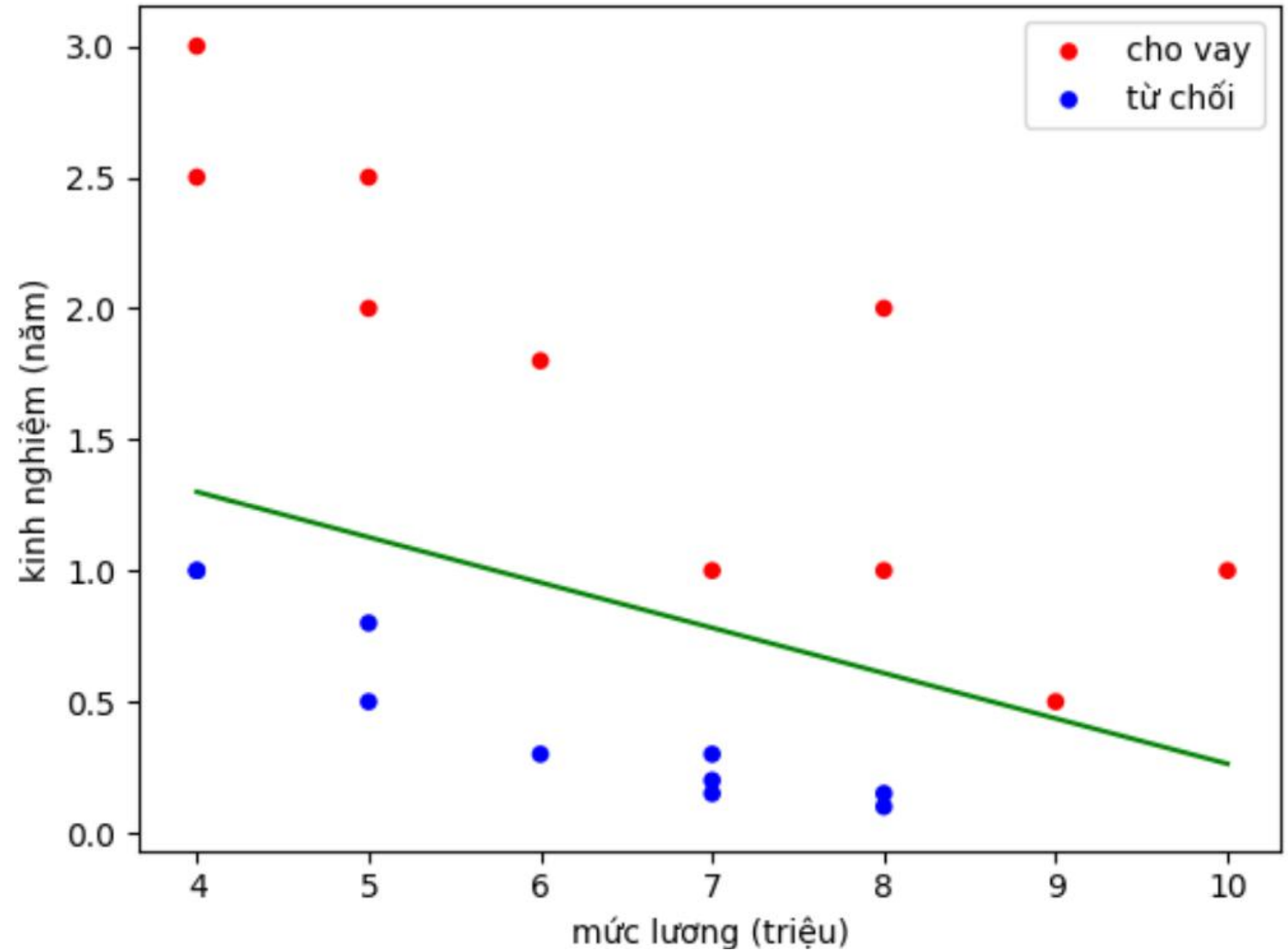
$$\text{Tương tự } \hat{y}_i < 0.5 \Leftrightarrow w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} < 0$$

$\Rightarrow$ đường thẳng $w_0 + w_1 x + w_2 y = 0$ chính là đường phân cách giữa các điểm cho vay và từ chối.

Trong trường hợp tổng quát bạn lấy xác suất lớn hơn t ($0 < t < 1$) thì mới cho vay tiền $\hat{y}_i > t \Leftrightarrow$
 $w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} > -\ln\left(\frac{1}{t} - 1\right)$

Quan hệ giữa phần trăm và đường thẳng

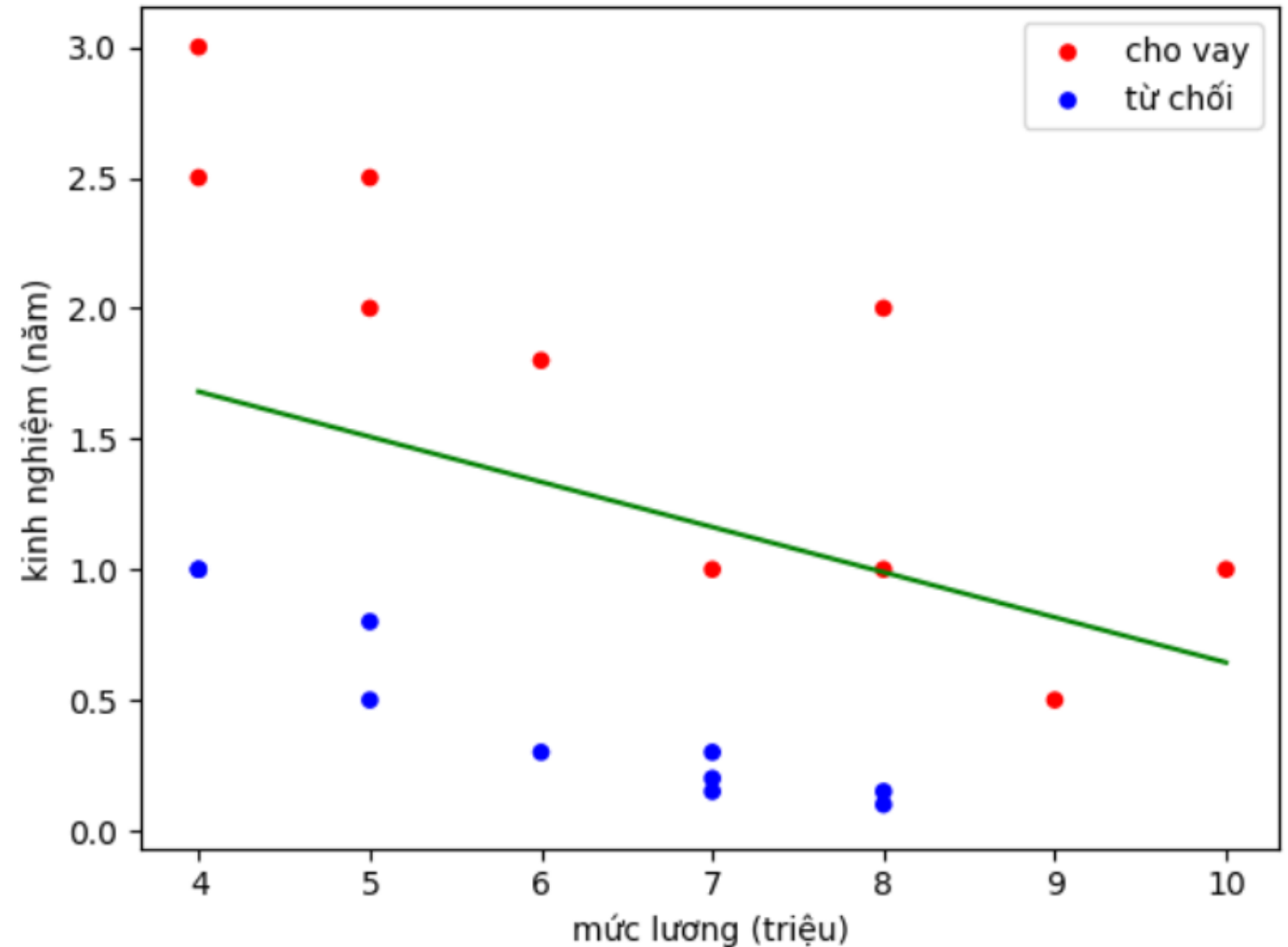
- Đường phân chia $t = 0.5$



Quan hệ giữa phần trăm và đường thẳng

- Đường phân chia $t = 0.8$

Ta thấy, đường phân chia gần với điểm màu đỏ hơn so với $t = 0.5$ thậm chí một số hồ sơ cũ trước được cho vay nhưng nếu giờ nộp lại cũng từ chối. Đồng nghĩa với việc công ty thắt chặt việc cho vay lại.



Quan hệ giữa phần trăm và đường thẳng

- Ứng dụng

- Spam detection: Dự đoán mail gửi đến hòm thư của bạn có phải spam hay không.
- Credit card fraud: Dự đoán giao dịch ngân hàng có phải gian lận không.
- Health: Dự đoán một u là u lành hay u ác tính.
- Banking: Dự đoán khoản vay có trả được hay không.
- Investment: Dự đoán khoản đầu tư vào startup có sinh lợi hay không.